



CME 295

Transformers & Large Language Models

Full Course Summary



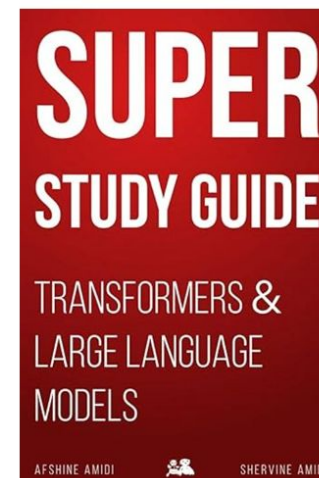
Afshine

Centrale Paris ('16), MIT ('17)
Uber, Google, Netflix



Shervine

Centrale Paris ('16), Stanford ('19)
Uber, Google, Netflix





Youtube : [Stanford CME295 Transformers & LLMs](#)

Course Materials : [Link](#)

Text Arabic Summary : [Link](#)

Course Prerequisites : ML, [intro to DL for NLP\(arabic\)](#)...

My LinkedIn : [Mohamed Mamdouh](#)



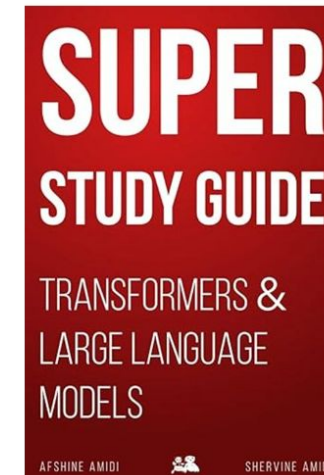
Afshine

Centrale Paris ('16), MIT ('17)
Uber, Google, Netflix



Shervine

Centrale Paris ('16), Stanford ('19)
Uber, Google, Netflix



Rewinding the quarter...

Lecture 1

NLP overview

Tokenization

Word representation

RNNs

Self-attention mechanism

Transformer architecture

End-to-end example

Lecture 2

Position embeddings

Layer normalization

Attention approximation

Transformer-based models

BERT deep dive

Lecture 3

LLM overview

MoE-based LLMs

Response generation

Prompting strategies

Inference optimizations

Lecture 4

Pretraining

Training optimizations

Supervised finetuning

Parameter-efficient finetuning

Lecture 5

Preference tuning

Data collection

RLHF

DPO

Lecture 6

Reasoning models

Scaling with RL

GRPO

Applications

Lecture 7

RAG

Tool calling

Agents

Lecture 8

Motivation

Rule-based metrics

LLM-as-a-Judge

Extensions

Benchmarks

Rewinding the quarter...

Lecture 1

Transformers

Rewinding the quarter...

Lecture 1

A cute teddy bear is reading.

Rewinding the quarter...

Lecture 1

A cute teddy bear is reading.

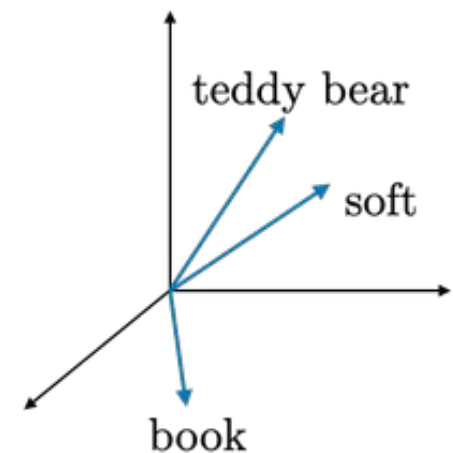
a cute teddy bear is reading .

Rewinding the quarter...

Lecture 1

A cute teddy bear is reading.

a cute teddy bear is reading .

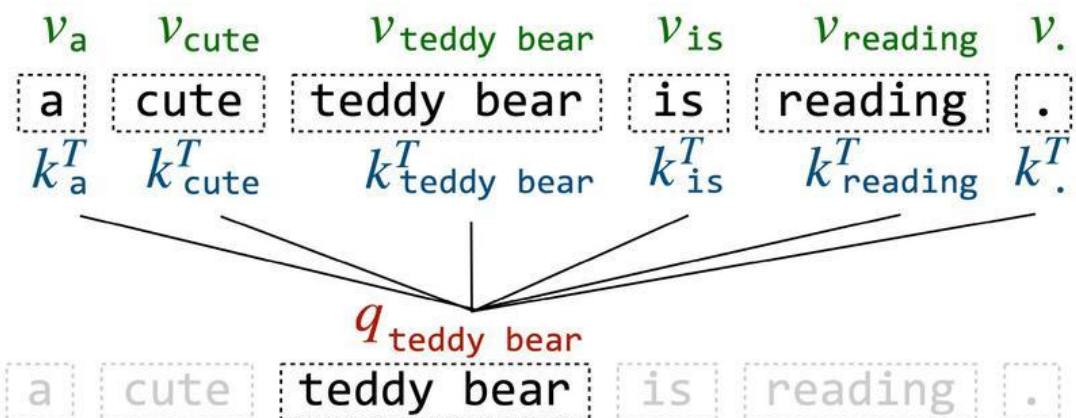


Rewinding the quarter...

Lecture 1

A cute teddy bear is reading.

a cute teddy bear is reading .



Rewinding the quarter...

Lecture 1

A cute teddy bear is reading.

a cute teddy bear is reading .

v_a v_{cute} $v_{\text{teddy bear}}$ v_{is} v_{reading} $v_{\text{.}}$
 a cute teddy bear is reading .
 k_a^T k_{cute}^T $k_{\text{teddy bear}}^T$ k_{is}^T k_{reading}^T $k_{\text{.}}^T$

$q_{\text{teddy bear}}$

a cute teddy bear is reading .



$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Rewinding the quarter...

Lecture 1

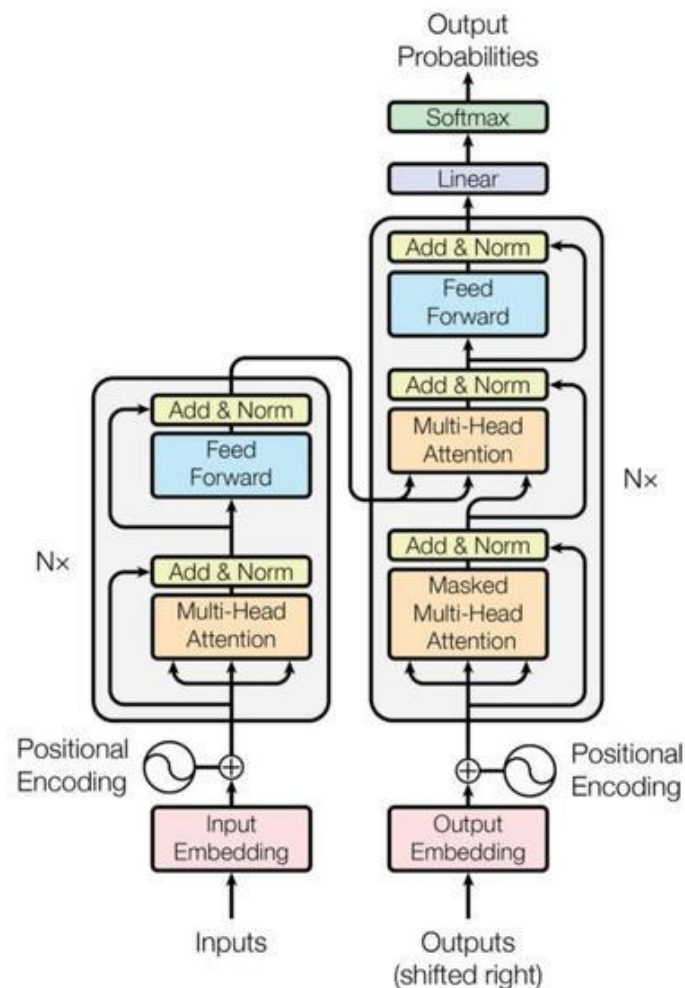
A cute teddy bear is reading.

a cute teddy bear is reading .

v_a v_{cute} $v_{\text{teddy bear}}$ v_{is} v_{reading} $v_{\text{.}}$
 a cute teddy bear is reading .
 k_a^T k_{cute}^T $k_{\text{teddy bear}}^T$ k_{is}^T k_{reading}^T $k_{\text{.}}^T$

a cute $q_{\text{teddy bear}}$ is reading .
 a cute teddy bear is reading .

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Rewinding the quarter...

Lecture 1

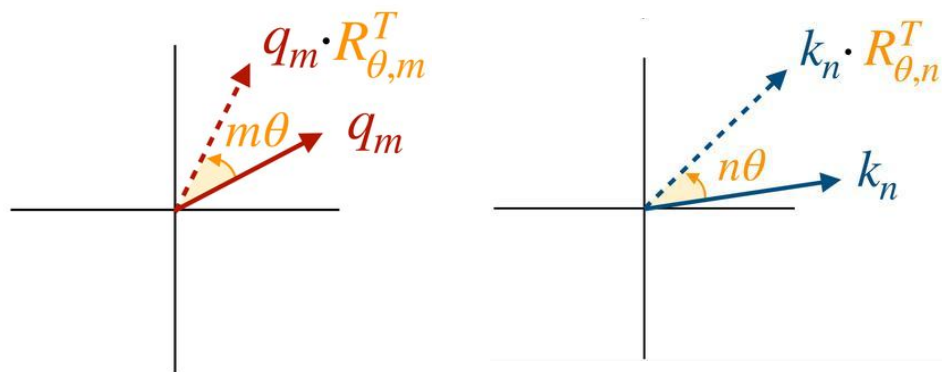
Lecture 2

Transformer-based models & tricks

Rewinding the quarter...

Lecture 1

Lecture 2



$$R_{\theta,m} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$

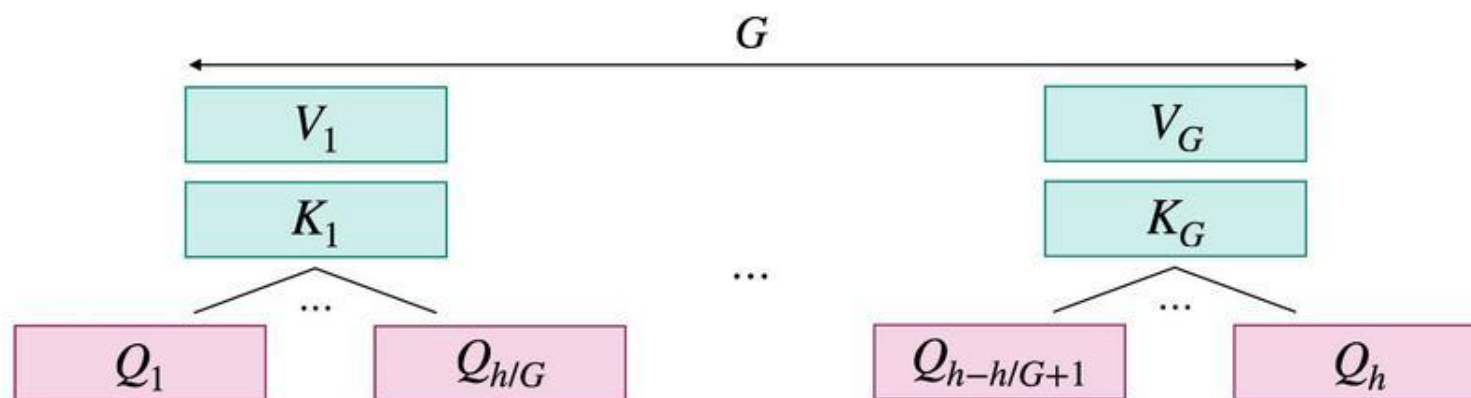
Rewinding the quarter...

Lecture 1

Lecture 2



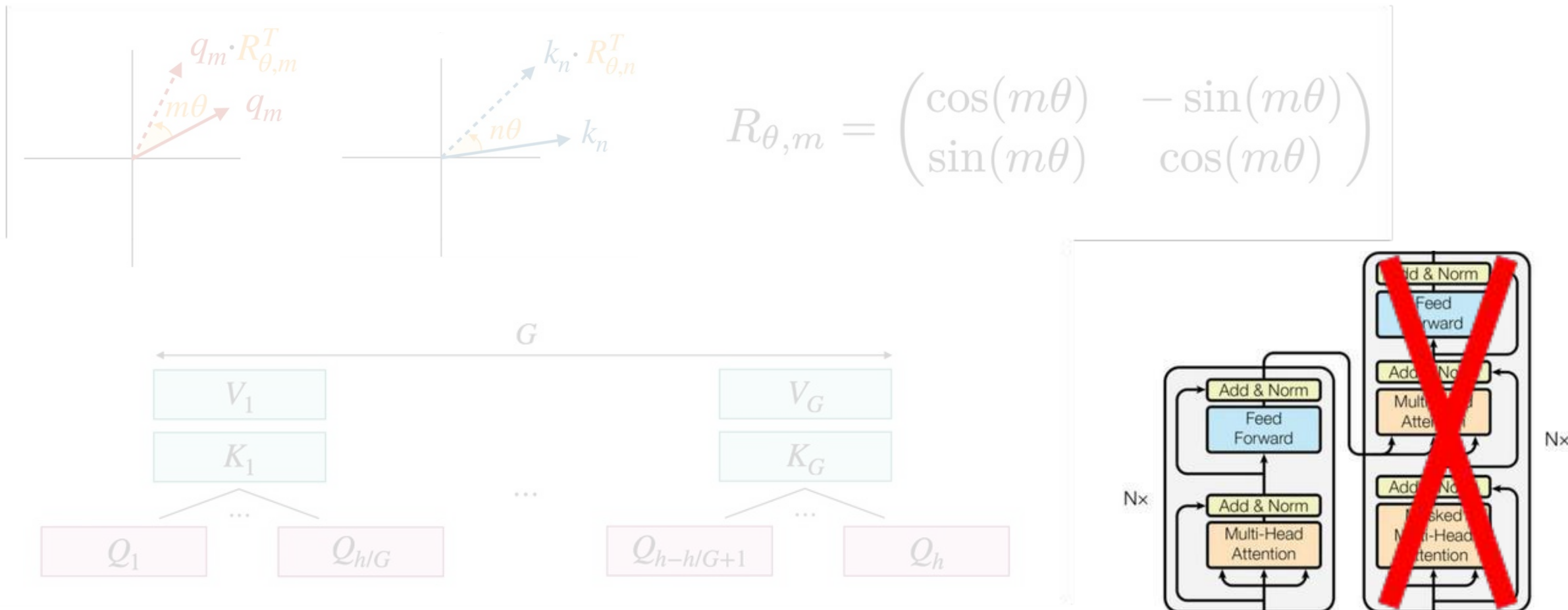
$$R_{\theta,m} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$



Rewinding the quarter...

Lecture 1

Lecture 2



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

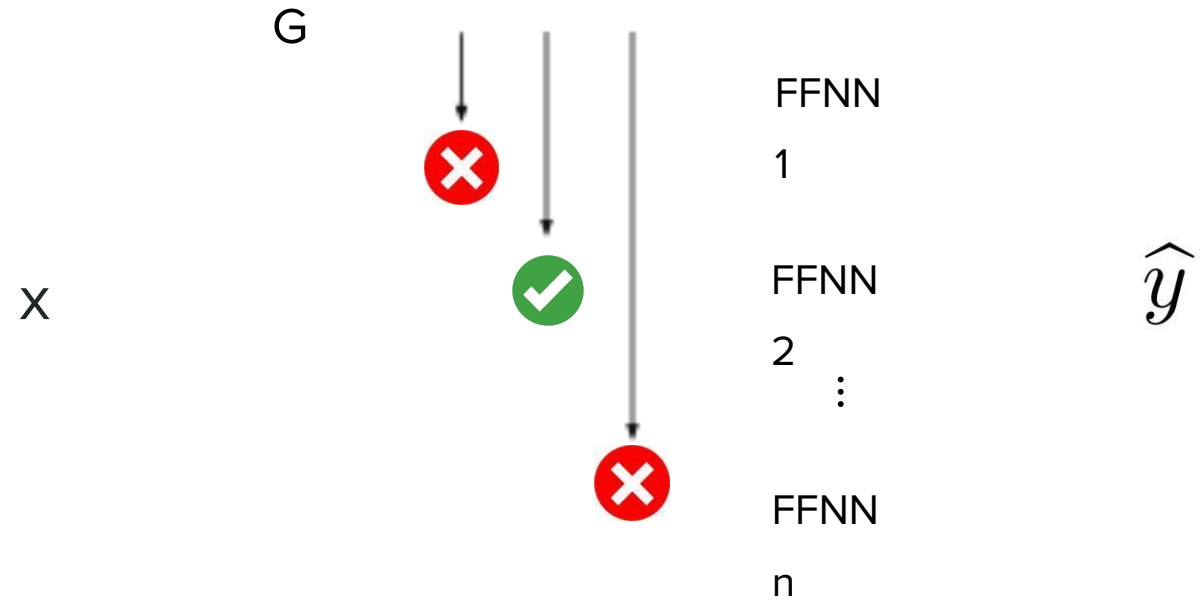
Large Language Models

Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Layer 0

```

class MoELayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                 gate: nn.Module, args):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token)
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                selected_experts == i)
            results[batch_idx] += weights[batch_idx, nth_expert] *
                self.experts[i].forward(inputs_squashed[batch_idx])
        return results.view_as(inputs)

```

G



FFNN

1

FFNN

2

⋮

FFNN

n

 $\hat{y}$

X

Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Layer 0

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
                  gate: nn.Module, args):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token)
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                selected_experts == i)
            results[batch_idx] += weights[batch_idx, nth_expert] *
                self.experts[i].forward(inputs_squashed[batch_idx])
        return results.view_as(inputs)
```

X

G



FFNN

1

FFNN

2

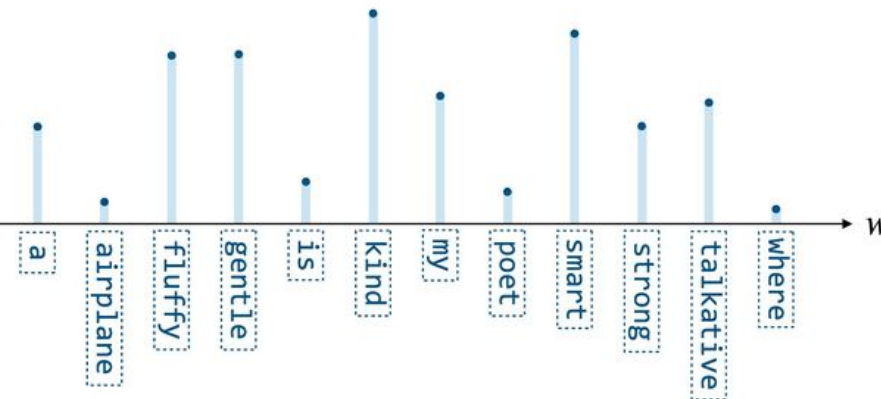
⋮

FFNN

n

 $\hat{y}$

$$P(w_{t+1} = w | C)$$



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

LLM training

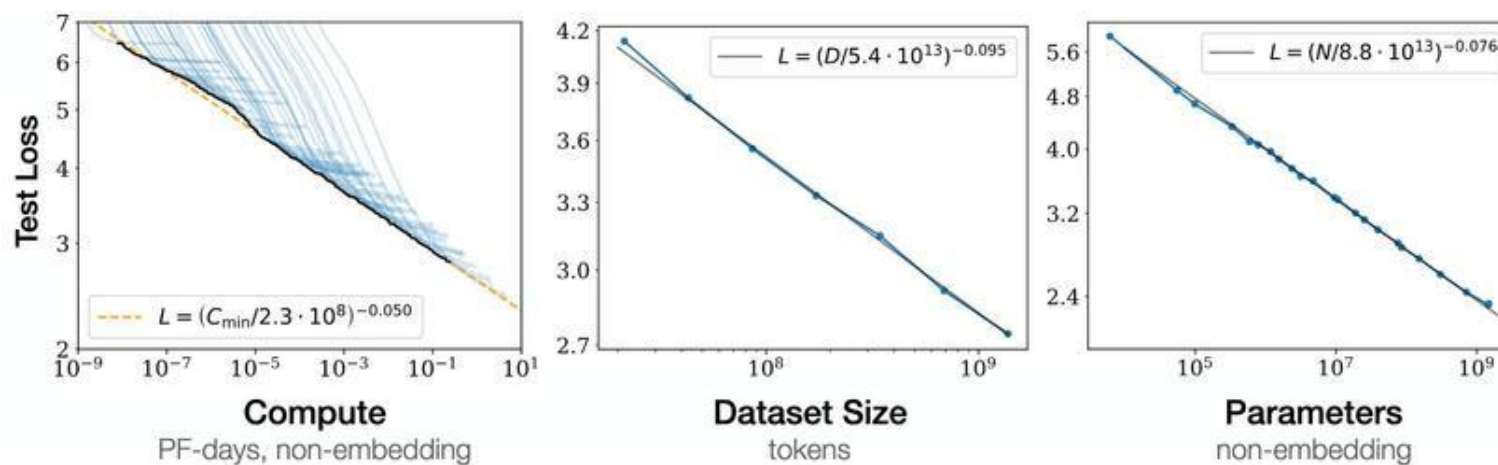
Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4



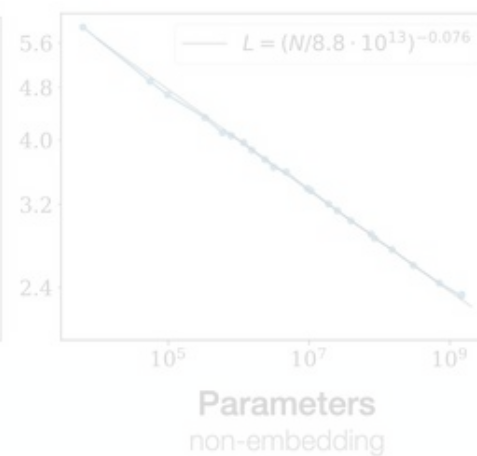
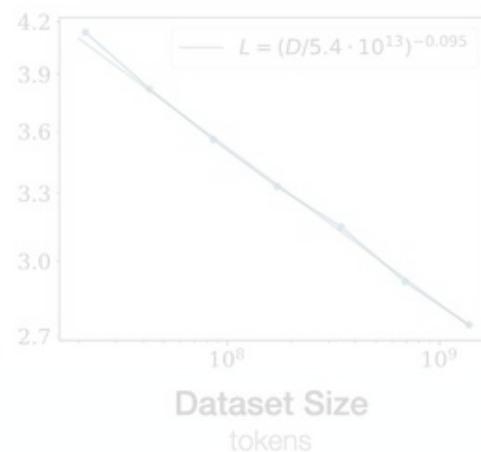
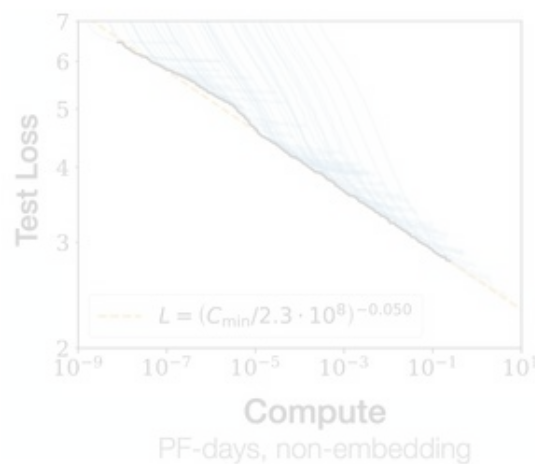
Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4



Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

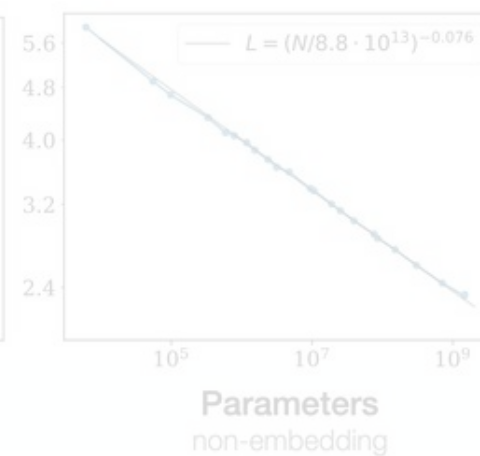
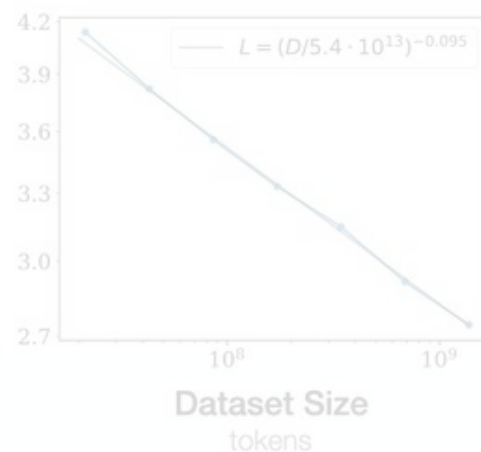
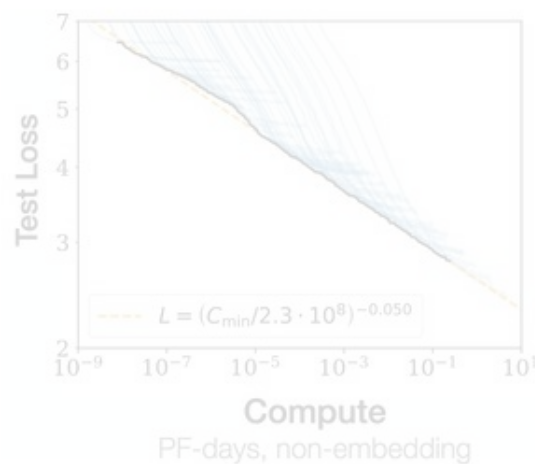
Rewinding the quarter...

Lecture 1

Lecture 2

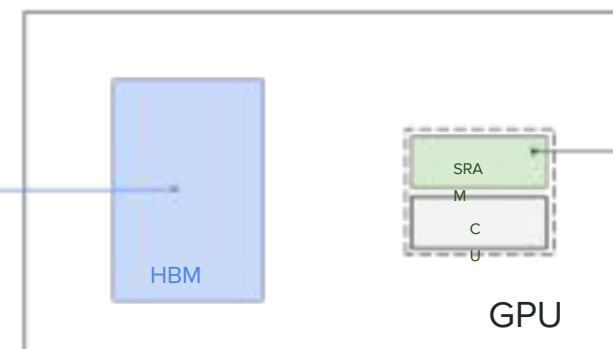
Lecture 3

Lecture 4



Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion

Big and slow



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Initialized model

Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Initialized model



Pretraining



Model with "basic
knowledge" about
language, code, etc.

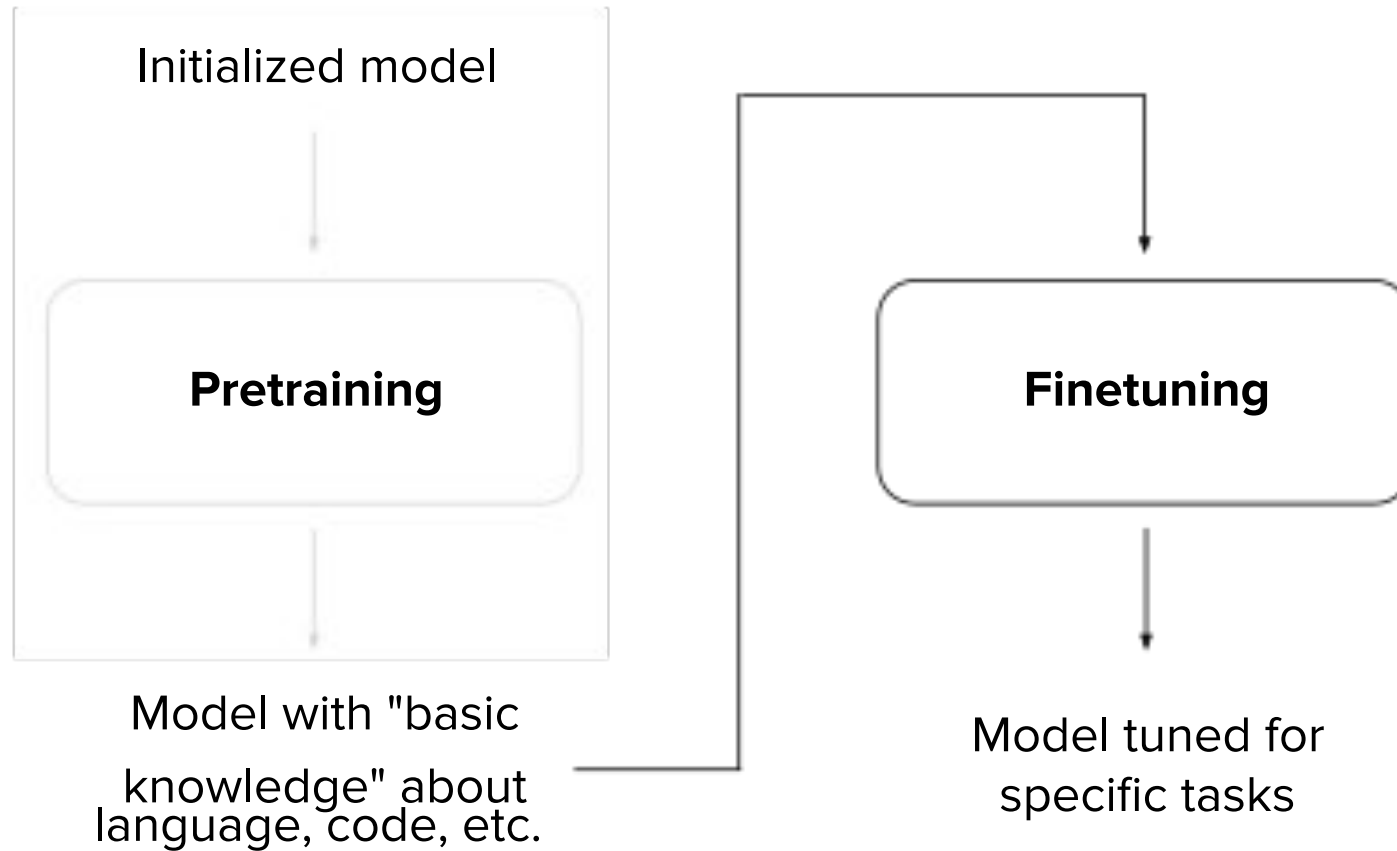
Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4



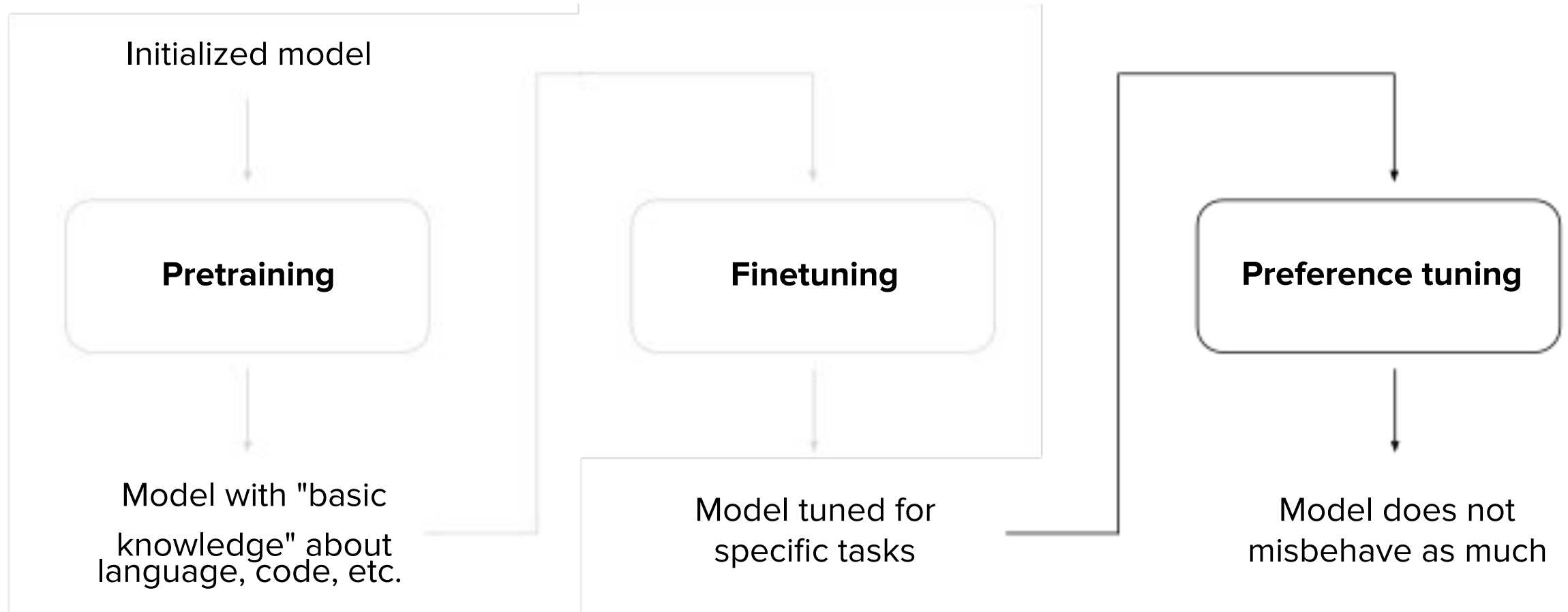
Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

LLM tuning

Rewinding the quarter...

Lecture 1

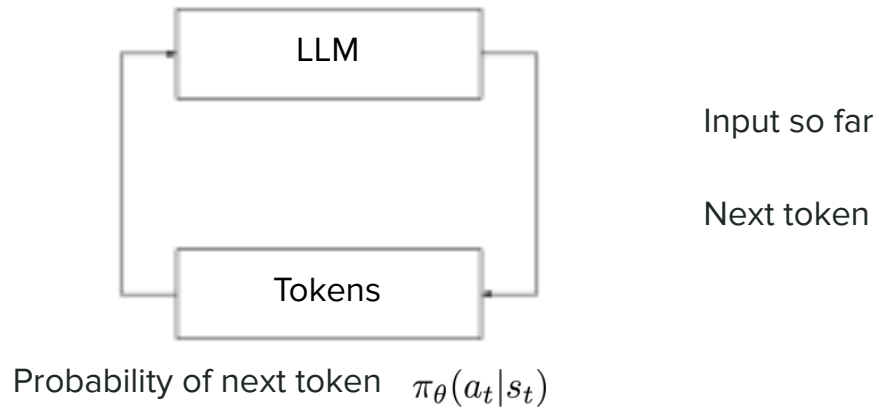
Lecture 2

Lecture 3

Lecture 4

Lecture 5

Human
preference r_t



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

Human
preference r_t



$$p(y_i > y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

Rewinding the quarter...

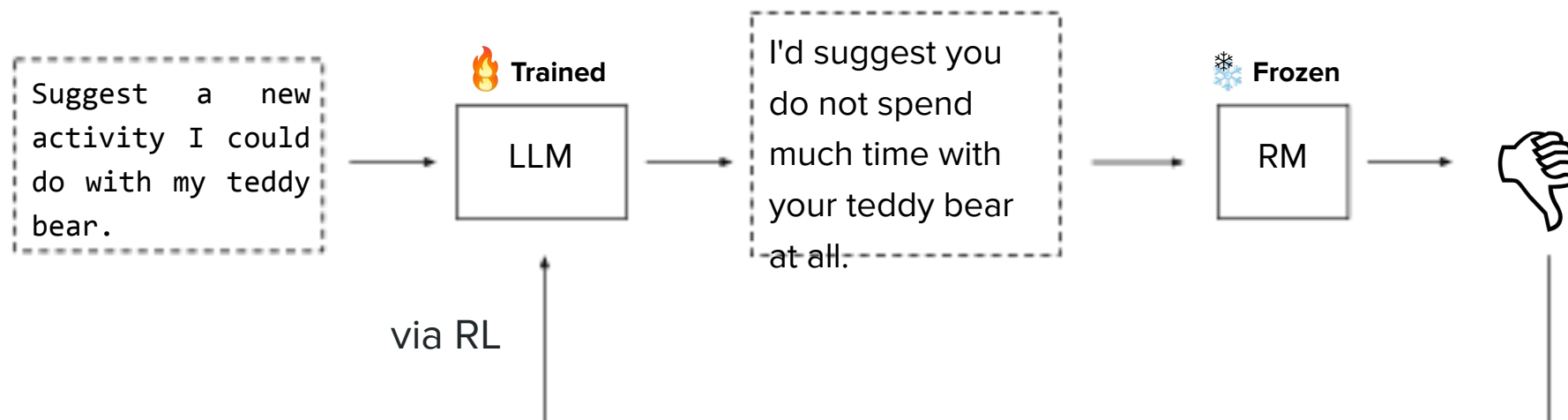
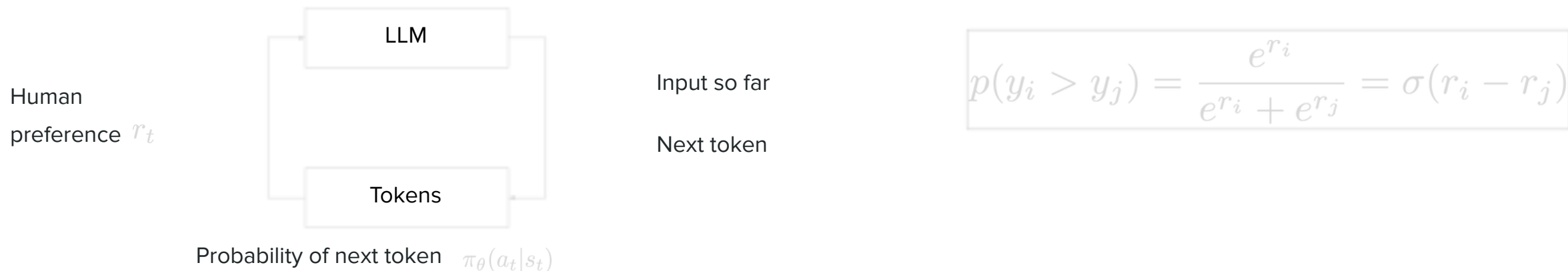
Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5



Rewinding the quarter...

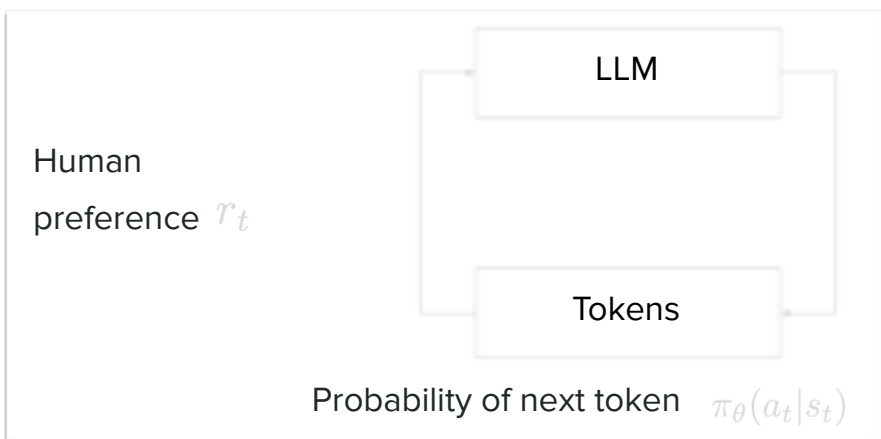
Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5



Input so far

Next token

$$p(y_i > y_j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j)$$

 $\mathcal{L}(\theta) =$

Maximize rewards

+

Don't deviate too much from old/base model



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

Lecture 6

LLM reasoning

Rewinding the quarter...

Lecture 1

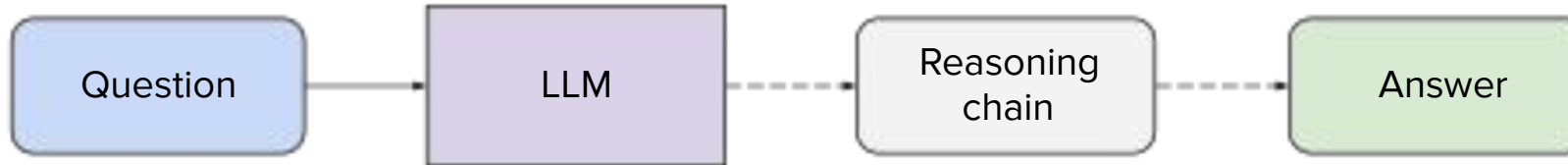
Lecture 2

Lecture 3

Lecture 4

Lecture 5

Lecture 6



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

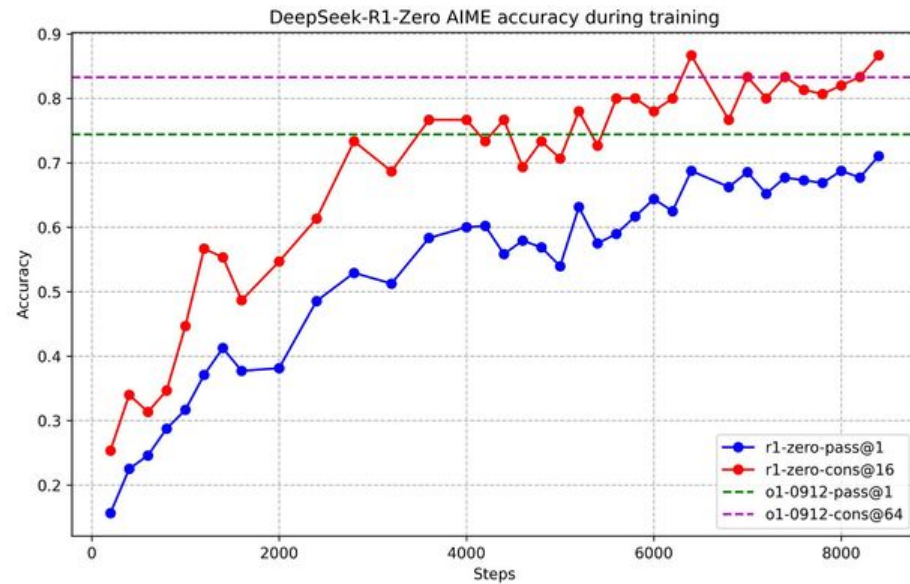
Lecture 6

Question

LLM

Reasoning
chain

Answer



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

Lecture 6

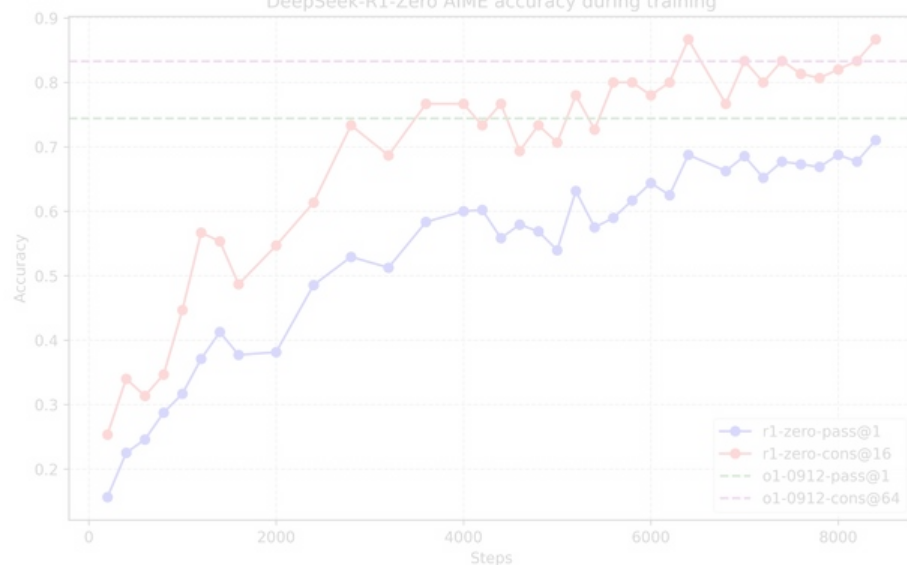
Question

LLM

Reasoning chain

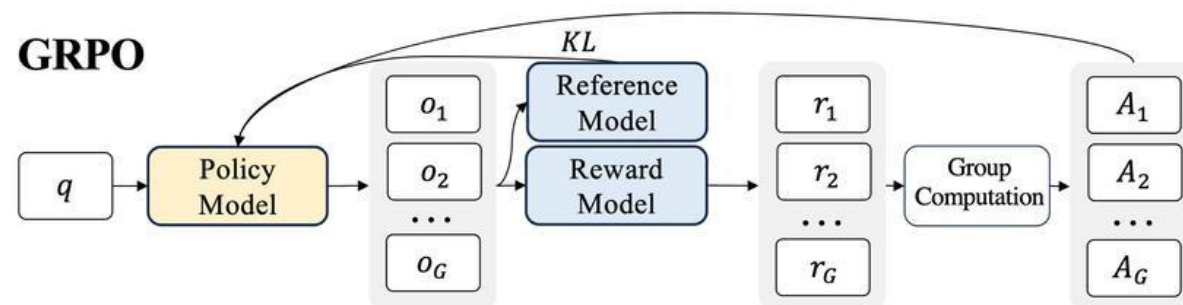
Answer

DeepSeek-R1-Zero AIME accuracy during training

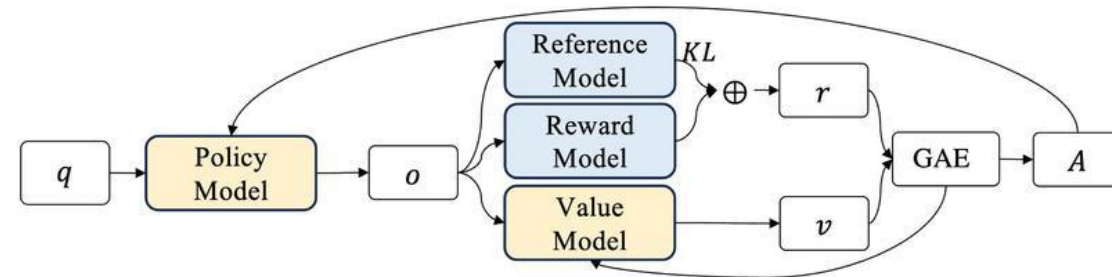


GRPO

GRPO



PPO



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

Lecture 6

Lecture 7

Agentic LLMs

Rewinding the quarter...

Lecture 1

Lecture 2

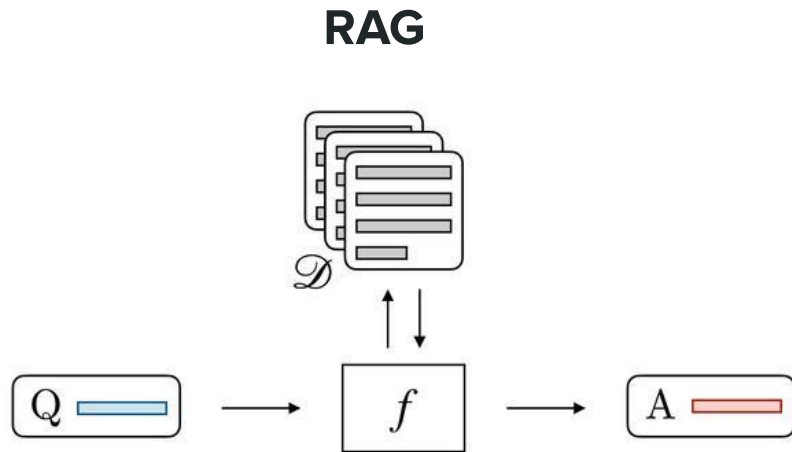
Lecture 3

Lecture 4

Lecture 5

Lecture 6

Lecture 7



Rewinding the quarter...

Lecture 1

Lecture 2

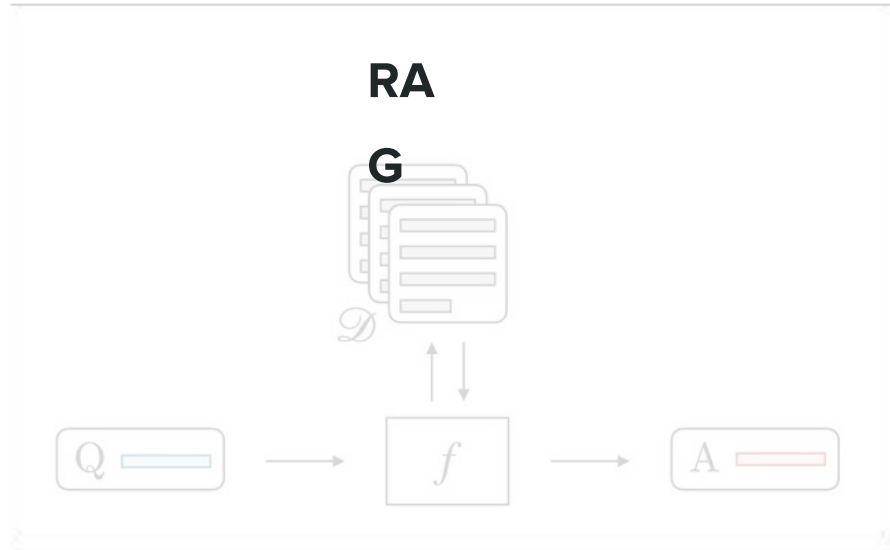
Lecture 3

Lecture 4

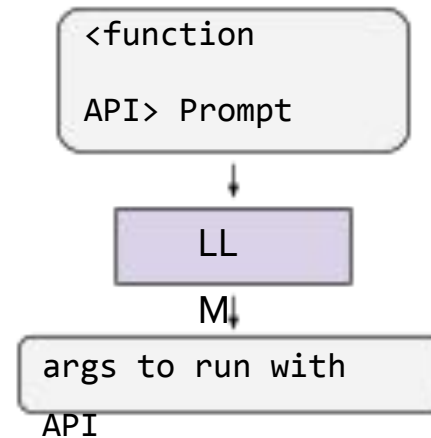
Lecture 5

Lecture 6

Lecture 7



Tool calling



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

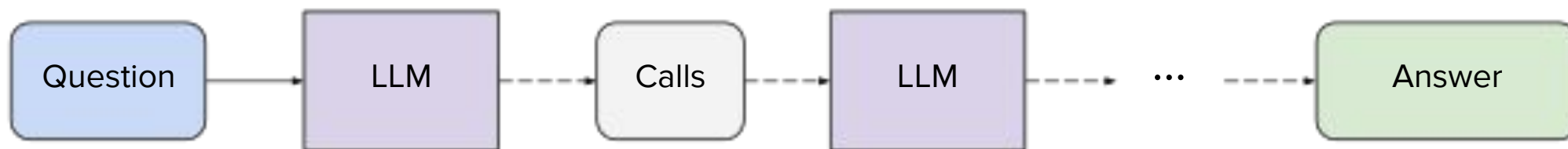
Lecture 5

Lecture 6

Lecture 7



Agent



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

Lecture 4

Lecture 5

Lecture 6

Lecture 7

Lecture 8

LLM evaluation

Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

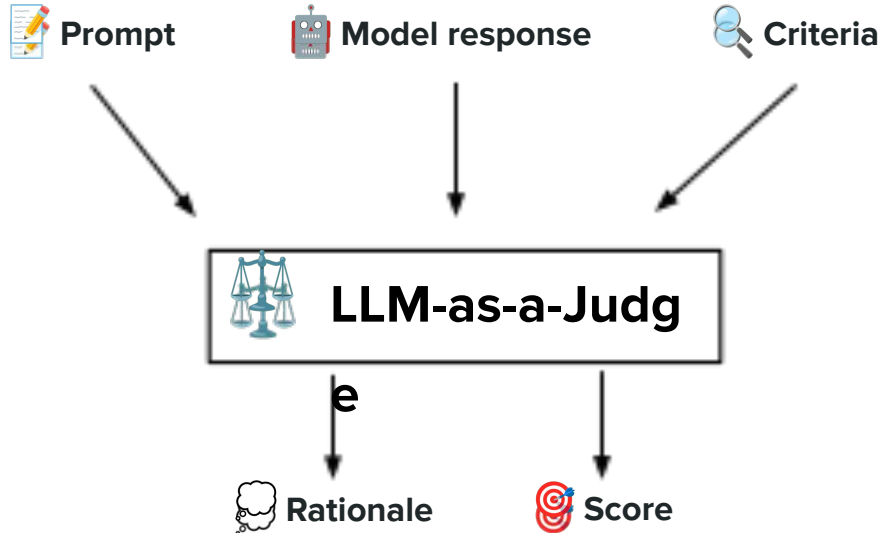
Lecture 4

Lecture 5

Lecture 6

Lecture 7

Lecture 8



Rewinding the quarter...

Lecture 1

Lecture 2

Lecture 3

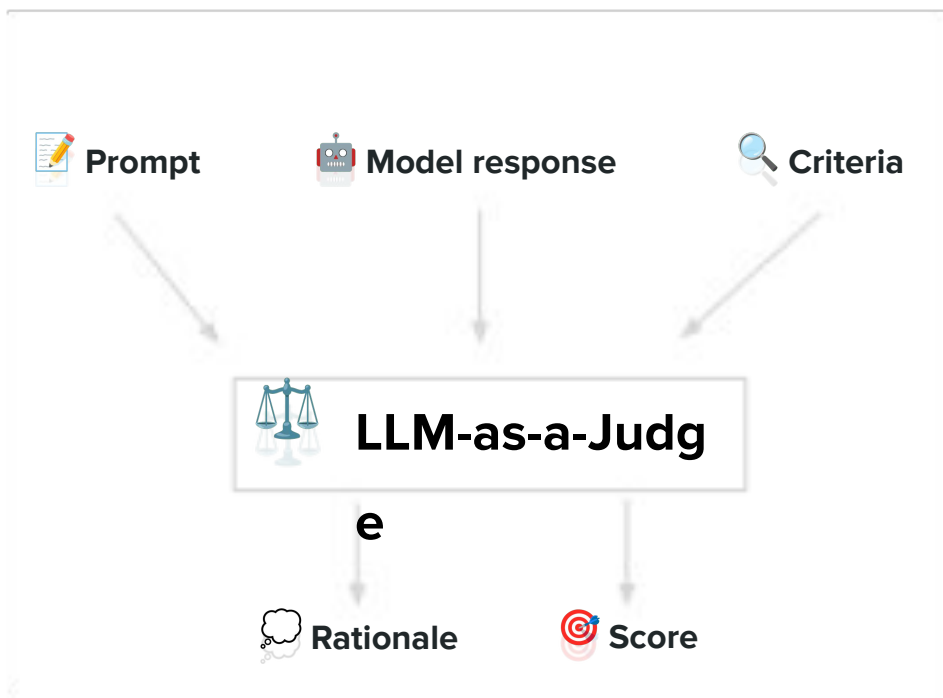
Lecture 4

Lecture 5

Lecture 6

Lecture 7

Lecture 8



Knowledge

- Accurately state facts about the world
- More "breadth" than "depth"
- Reflects pretraining quality

MMLU

Reasoning

- Solve multi-step problems
- Includes both "maths" and "common sense" subfields
- Internal reasoning

AIME

PIQA

Coding

- Generate syntactically correct code
- Tests for programming proficiency
- Proxy for tool use abilities

SWE-bench

Safety

- Prevent harmful, toxic, inappropriate behavior
- Surfaces vulnerabilities before deployment
- Alignment with custom preferences

HarmBench

Course roadmap

A transformer course is a sequence of abstractions: each layer turns the previous one into a more powerful interface.



Key idea: The course moves from mathematical building blocks to deployable, evaluated AI systems.

Tokenization: text becomes model-readable symbols

Tokenizers split raw text into reusable units and map each unit to an integer ID.



Why it matters

Vocabulary size controls
compression vs. flexibility

Subwords handle rare
and unseen words

Token count sets cost,
latency, and context use

Key idea: The model never sees characters directly; it sees token IDs that index learned vectors.

Embeddings: tokens become vectors

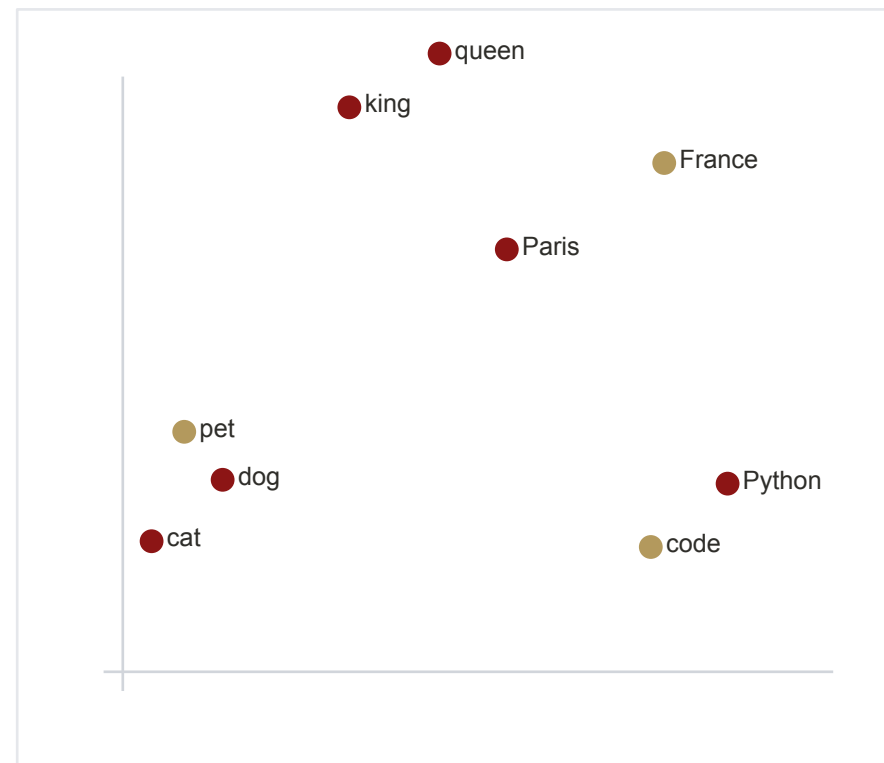
An embedding table turns token IDs into dense vectors whose geometry encodes learned similarity and usage.

Token ID	Vector
464	[0.12, -0.08, ..., 0.31]
2068	[-0.44, 0.19, ..., 0.05]
21831	[0.02, 0.77, ..., -0.16]



**Embeddings are
learned parameters**

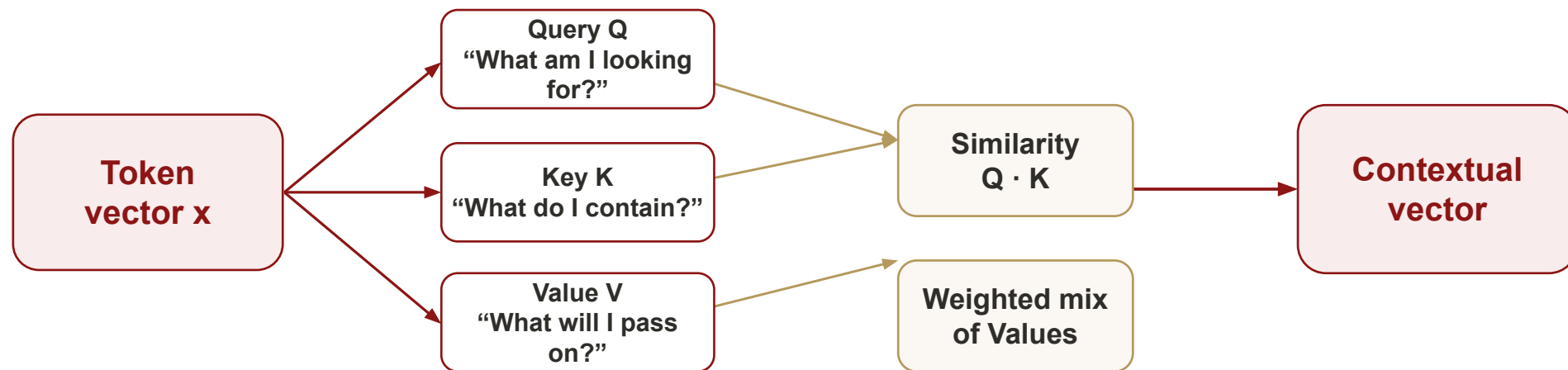
semantic neighborhood (conceptual)



Key idea: Vectors let the model compute with meaning-like relationships using linear algebra.

Attention: Query, Key, Value

Self-attention asks: “Which earlier or neighboring tokens should this token read from?”

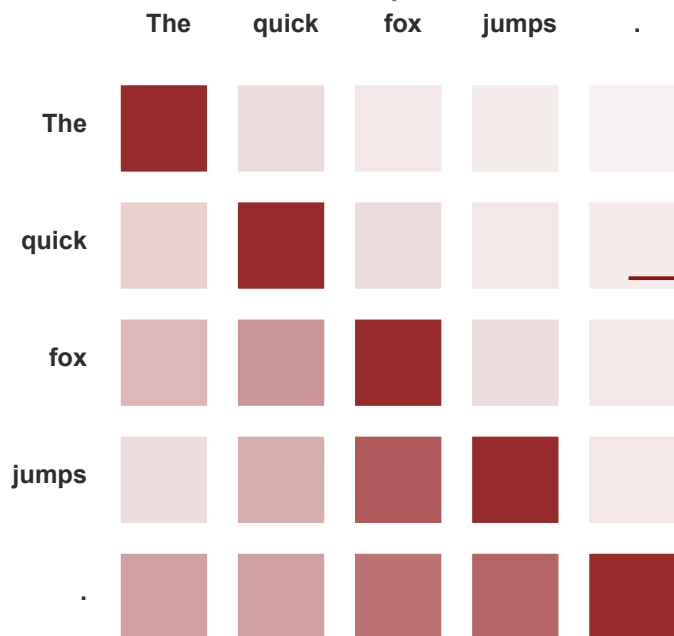


$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Key idea: Attention dynamically routes information between tokens instead of using a fixed local window.

Self-attention: every token can read the context

The attention matrix stores how strongly each output token attends to each input token.



Masked in decoder models:
future tokens are hidden

Attention
weights

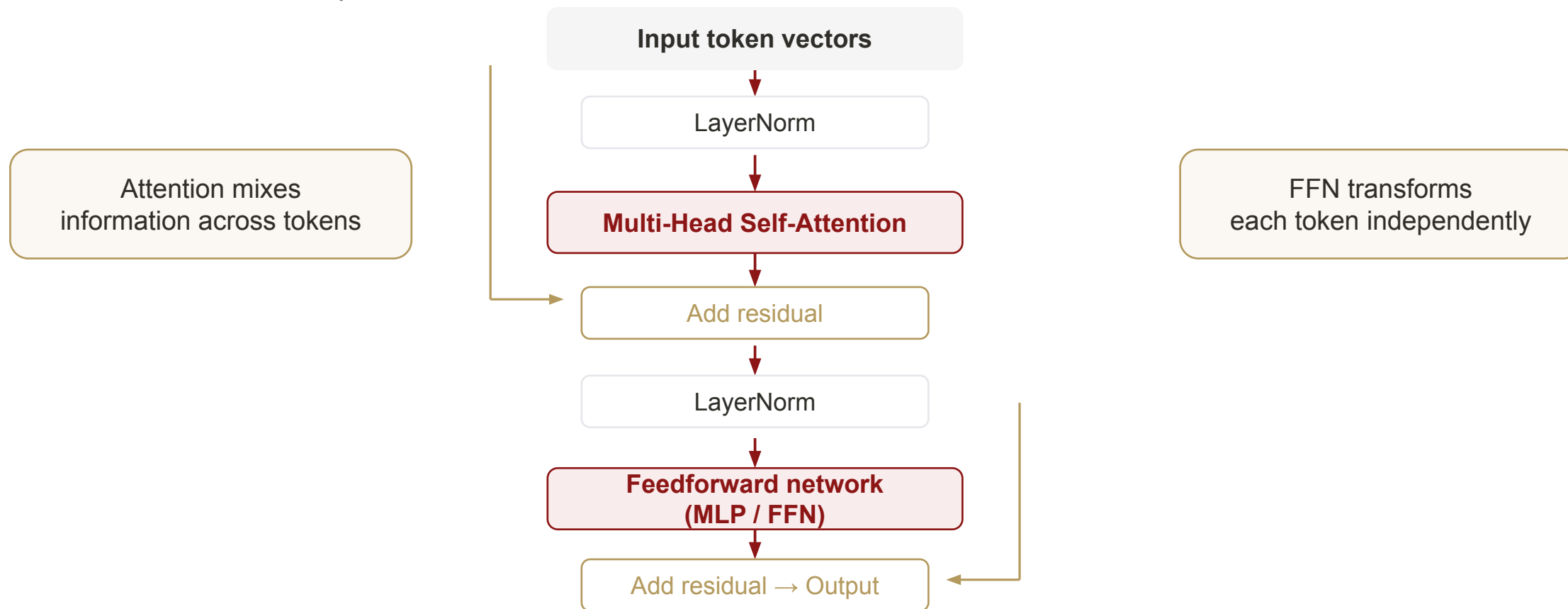
Weighted sum
of value vectors

Contextual
outputs

Key idea: Self-attention creates context-aware token vectors by mixing information across the sequence.

Transformer block: attention + feedforward + residuals

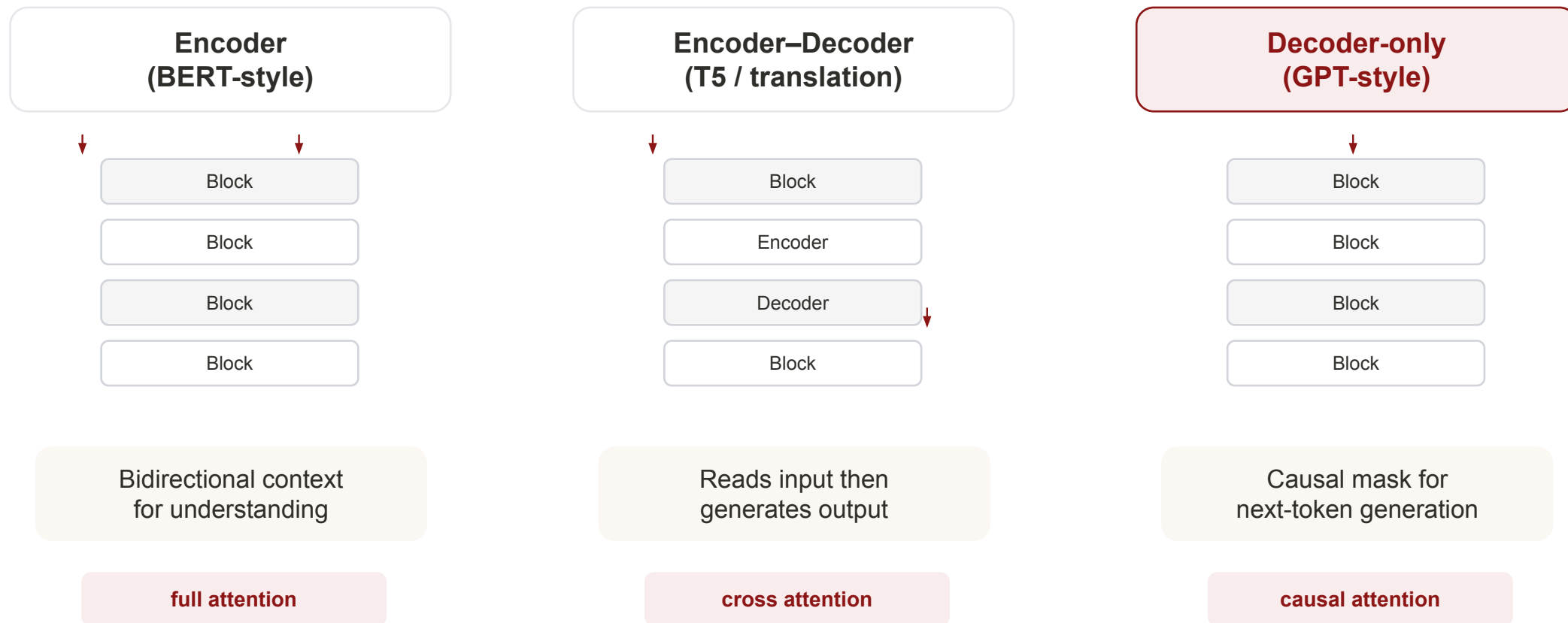
A transformer is a repeated stack of blocks that alternate communication and computation.



Key idea: Residual paths and normalization keep deep stacks trainable.

Encoder, encoder-decoder, and decoder-only models

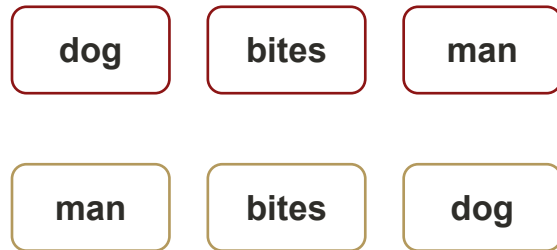
The attention mask and data flow determine which tasks a transformer is naturally suited for.



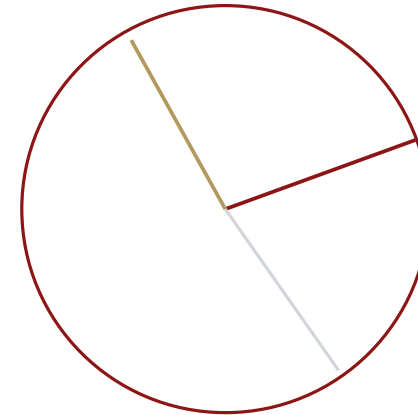
Key idea: Most modern chat LLMs are decoder-only because the training and serving loop is simple: predict the next token.

Position matters: positional encoding and RoPE

Attention alone is permutation-invariant; models need position information to understand order.



**same tokens,
different meaning**



**RoPE rotates Q/K
by position-dependent angles**

Absolute positions:
add or learn a vector

Relative position:
attention sees distance

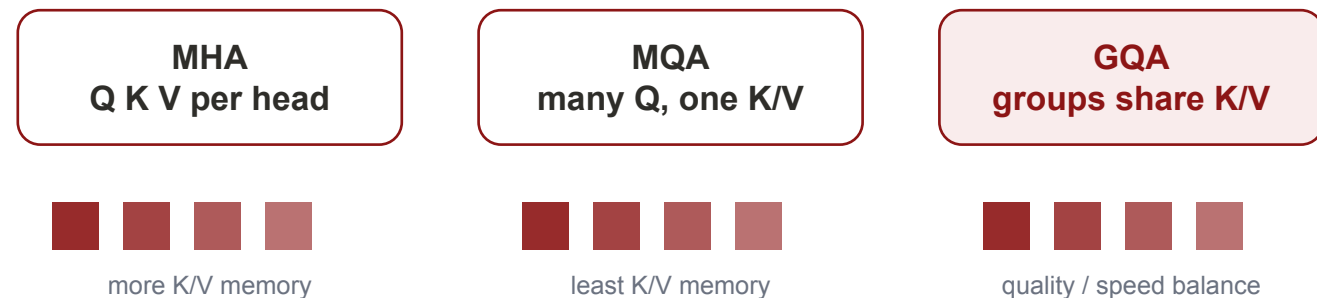
RoPE:
relative distance via rotations

Key idea: RoPE encodes relative order elegantly by rotating query and key vectors.

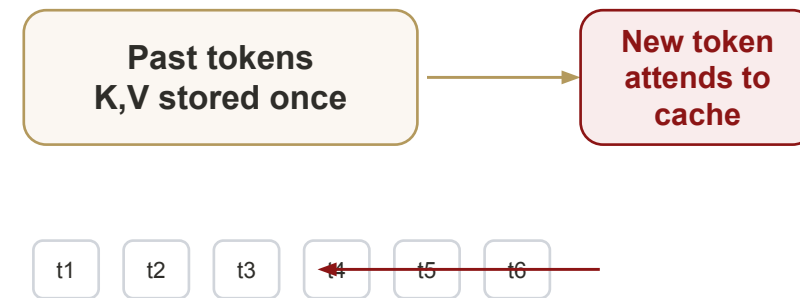
Attention optimizations for efficient inference

Serving LLMs is dominated by memory movement and repeated attention over growing context.

HEAD SHARING



KV CACHE

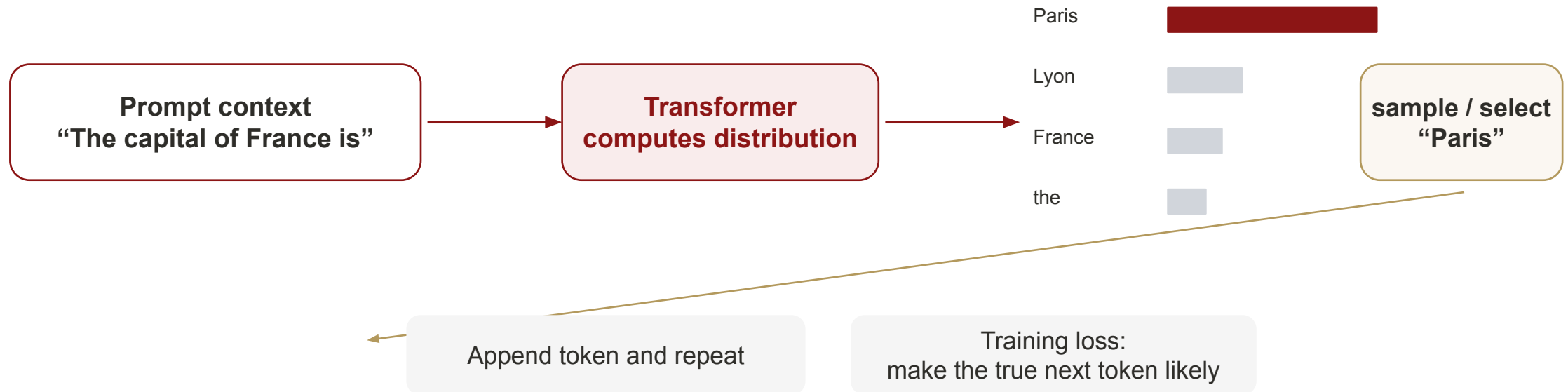


Caching avoids recomputing old K/V every step

Key idea: MHA, MQA, GQA, and KV caching trade memory, quality, and latency during generation.

Large Language Models: next-token prediction

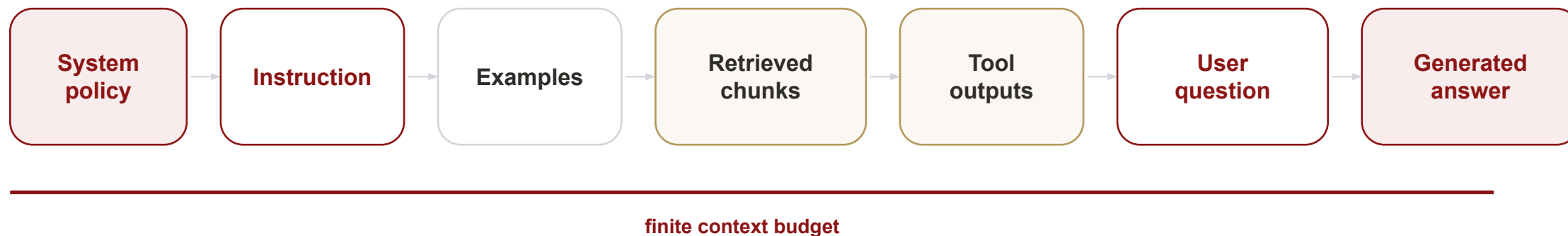
Autoregressive LLMs learn to predict the next token, then repeat that step to generate text.



Key idea: Complex behavior emerges from repeatedly applying a simple conditional prediction task.

Context window: the model's working memory

The prompt, examples, retrieved text, tool results, and conversation history all compete for finite context space.



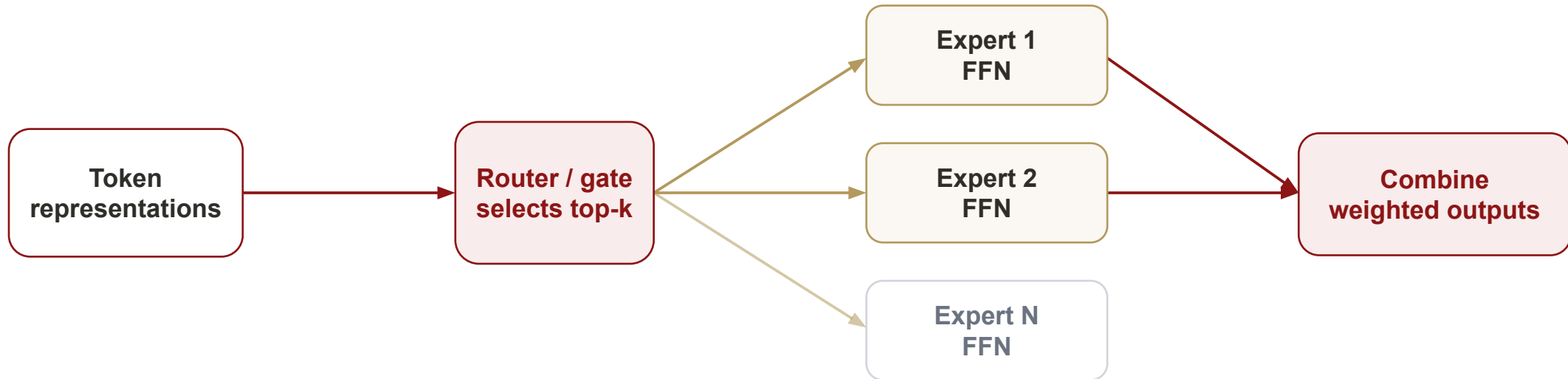
In-context learning:
examples steer behavior without changing weights

Long context helps, but retrieval quality and attention
patterns still matter

Key idea: Good prompting is context design: place the right evidence and instructions where the model can use them.

Mixture of Experts: sparse capacity at scale

MoE models route each token to a small subset of expert FFNs, increasing parameters without using all of them every token.



Benefits

- more capacity
- lower active compute
- specialization

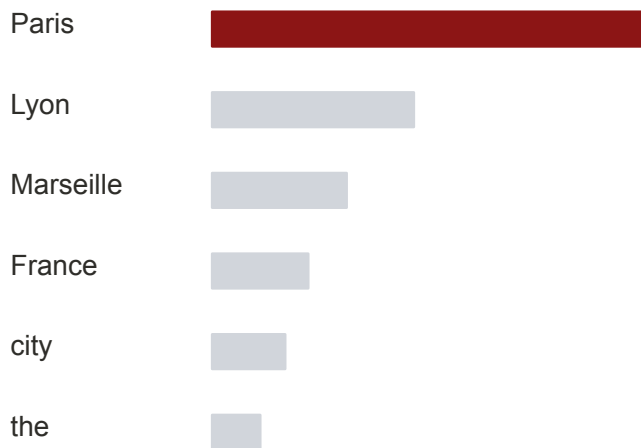
Challenges

- load balancing
- routing stability
- distributed serving

Key idea: MoE separates total parameters from active parameters.

Generation strategies: choosing the next token

Decoding turns a probability distribution into actual text, controlling determinism, diversity, and risk.



**Greedy
choose max**

**Temperature
reshape confidence**

**Top-k
keep k options**

**Top-p
keep probability
mass**

low T

high T

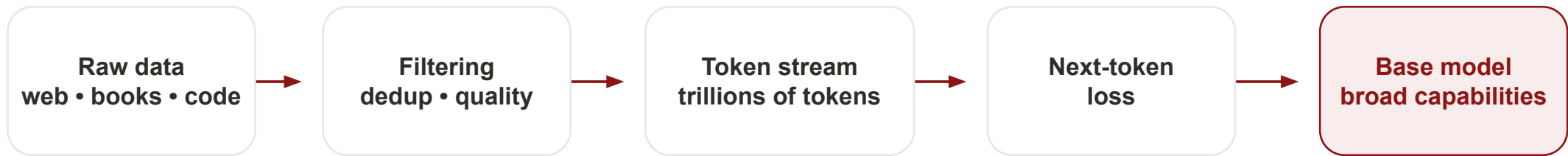
Deterministic
precise

Diverse
creative

Key idea: Decoding is a product choice: optimize for factuality, creativity, latency, or safety.

Pretraining: learning broad language patterns at scale

Base LLMs are trained on massive token streams with a simple objective: predict the next token.



Objective:
minimize cross-entropy on true next token

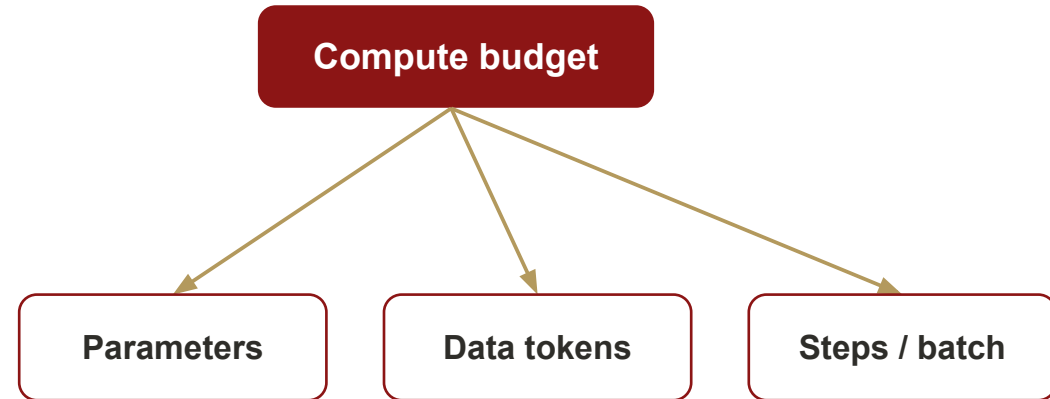
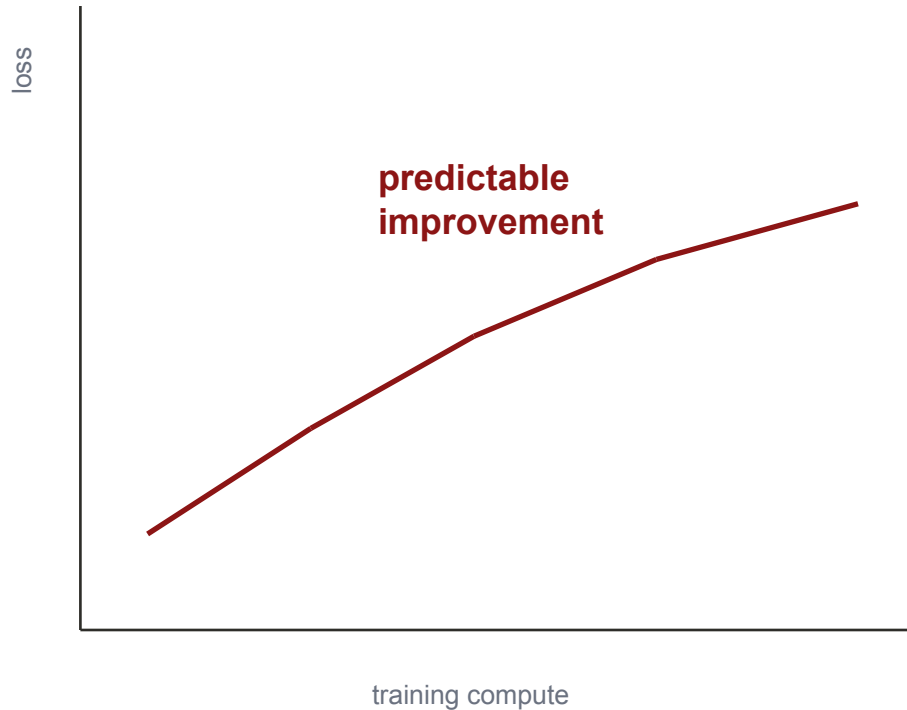
Data quality matters:
coverage, contamination, bias, recency

Pretraining creates a base
model,
not yet a helpful assistant

Key idea: Pretraining teaches general statistical structure; later stages shape behavior.

Scaling laws and compute-optimal training

Performance improves predictably with more compute, parameters, and data—but the best allocation matters.



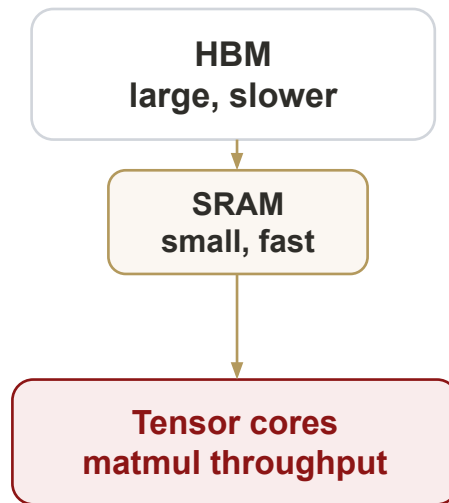
Too few tokens → undertrained model
Too many parameters → inefficient use of compute

Key idea: Compute-optimal training balances model size and dataset size for the budget.

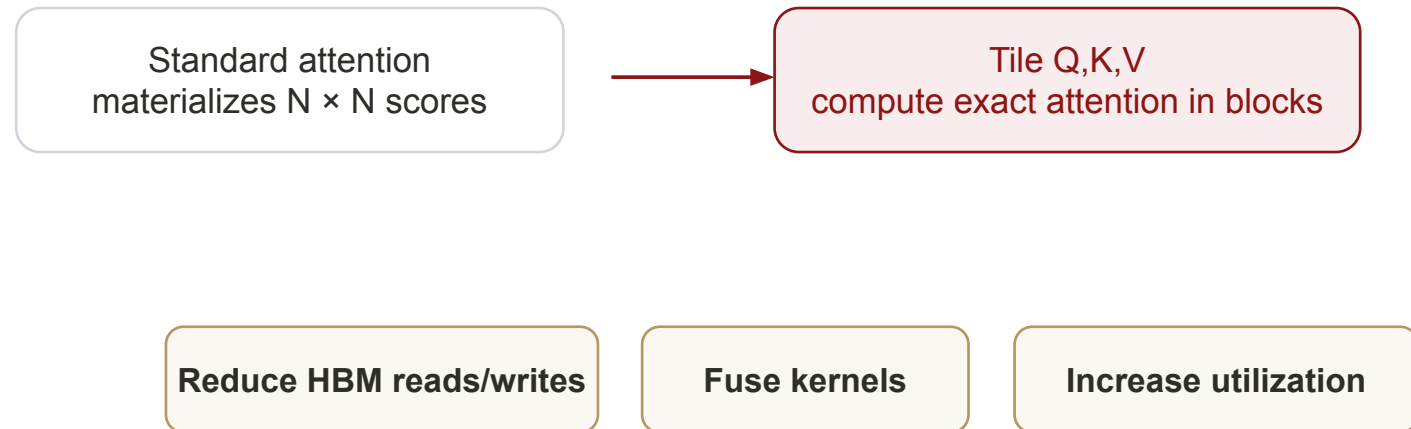
Training optimizations: make GPUs do useful work

Modern training speed depends as much on memory traffic and kernels as on raw FLOPs.

MEMORY HIERARCHY



FLASHATTENTION IDEA

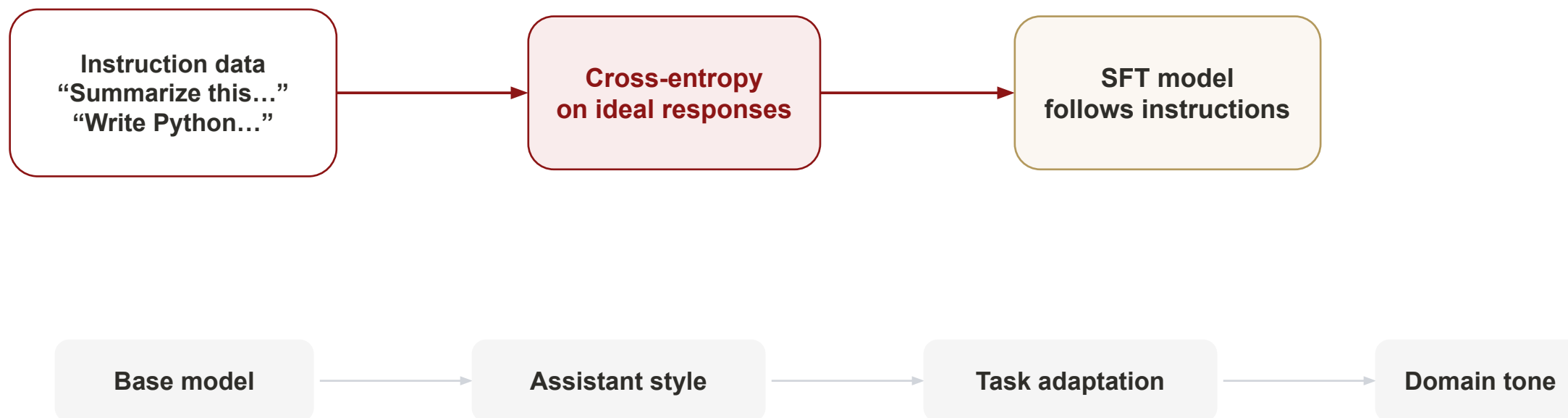


System optimizations also include mixed precision, activation checkpointing, distributed parallelism, and efficient kernels.

Key idea: The fastest model is often the one that moves less data, not just the one with fewer operations.

Supervised fine-tuning: turn a base model into an assistant

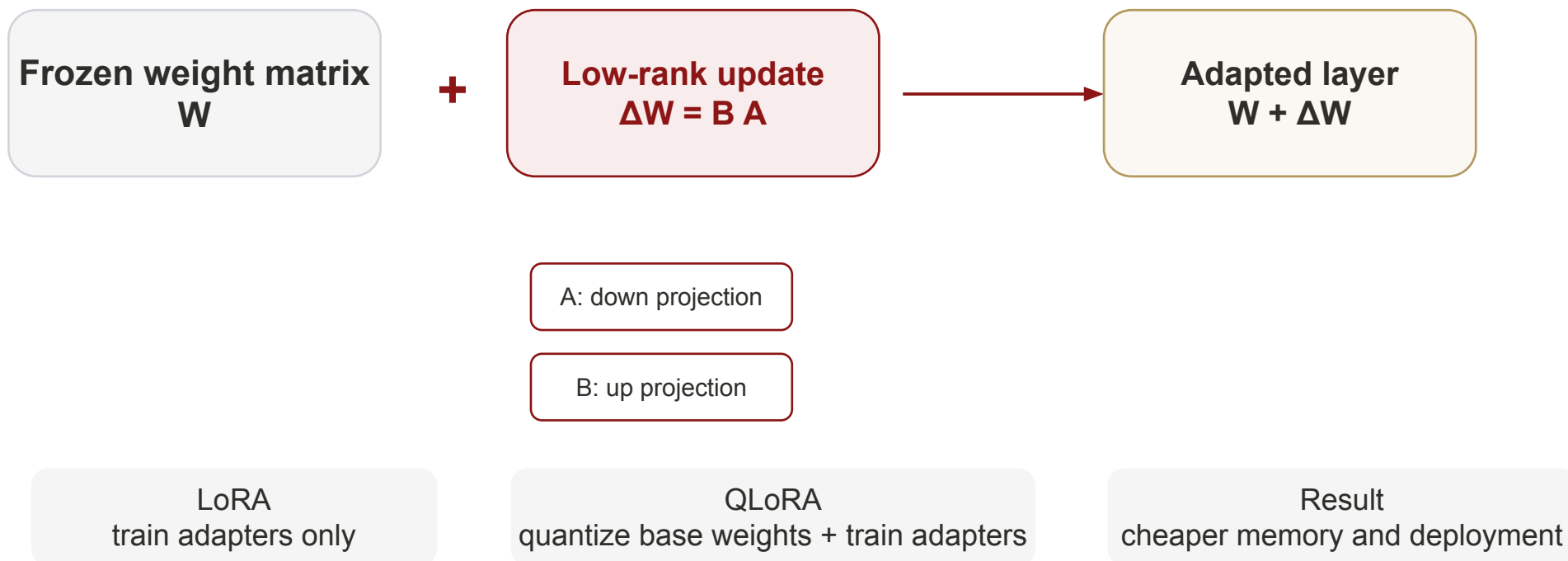
SFT trains the model on instruction–response examples so it learns the interaction format and desired tasks.



Key idea: SFT teaches “how to answer,” but not necessarily which answer humans prefer.

Parameter-efficient fine-tuning: LoRA and QLoRA

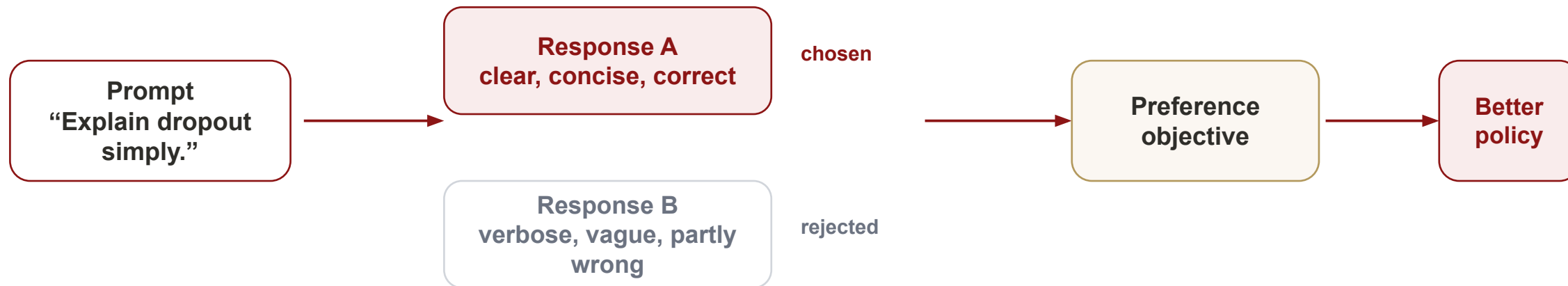
PEFT adapts a large model by training small extra modules while keeping most base weights frozen.



Key idea: LoRA changes the update, not the whole model; QLoRA adds quantization for lower memory.

Preference tuning: learn from chosen vs. rejected answers

Preference data tells the model which of two plausible responses better matches human intent and norms.

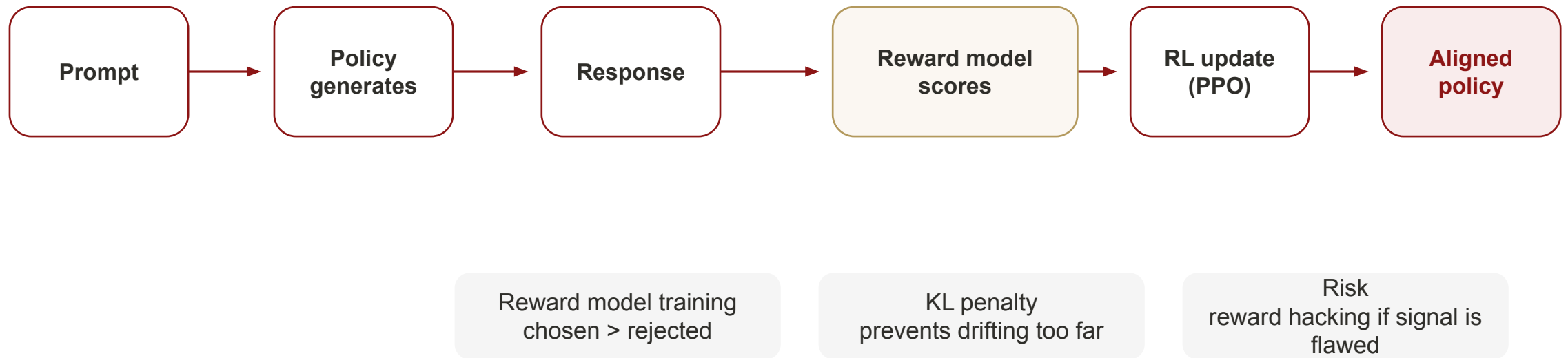


Preferences capture style, helpfulness, harmlessness, and task quality when labels are well-designed.

Key idea: Preference tuning optimizes relative quality, not just imitation of a single target answer.

RLHF: reward models turn preferences into an optimization signal

RLHF trains a policy to generate responses that score well under a learned reward model while staying close to the reference model.



Key idea: RLHF separates “judge quality” from “generate text,” then optimizes the generator against the judge.

PPO vs. DPO: two routes from preferences to policy

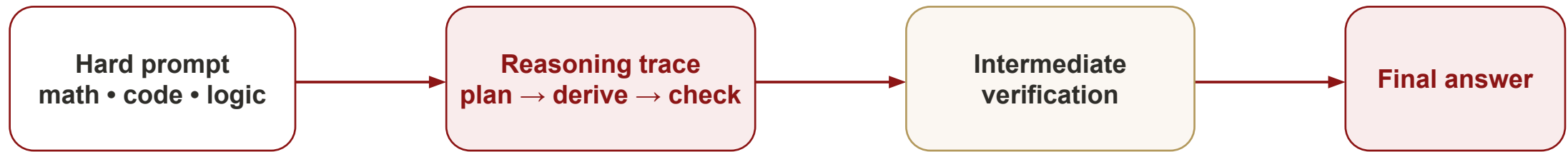
Both methods use preference data; they differ in whether they explicitly train and optimize a reward model.

Dimension	PPO-style RLHF	DPO
Main idea	Optimize policy against learned reward model	Directly optimize preference classification objective
Pipeline	Reward model + RL loop + KL control	No explicit reward model in the standard formulation
Strength	Flexible reward shaping and online sampling	Simpler, stable, computationally lighter
Challenge	Sensitive and complex to tune	Depends strongly on preference data quality

Key idea: DPO is a simpler preference-tuning path; PPO-style RLHF is more flexible but operationally heavier.

Reasoning models: train the model to spend computation on thinking

Reasoning models improve difficult tasks by learning to decompose, verify, and revise before answering.



Chain-of-thought prompting
encourages decomposition

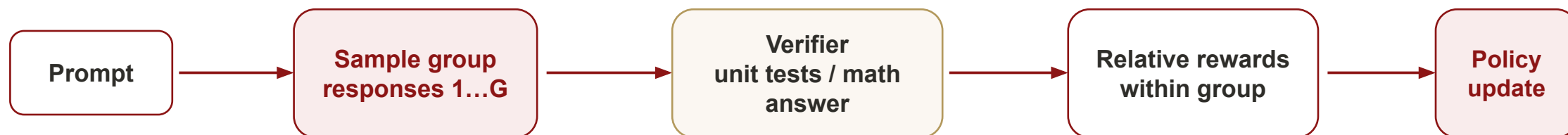
Reasoning training
rewards processes or outcomes

Inference-time compute
can trade latency for accuracy

Key idea: Reasoning quality improves when models learn to use steps, checks, and search—not only memorized patterns.

Verifiable rewards and GRPO

For math and code, correctness can often be checked automatically, enabling outcome-based reinforcement learning.



Verifiable rewards
reduce reliance on subjective labels

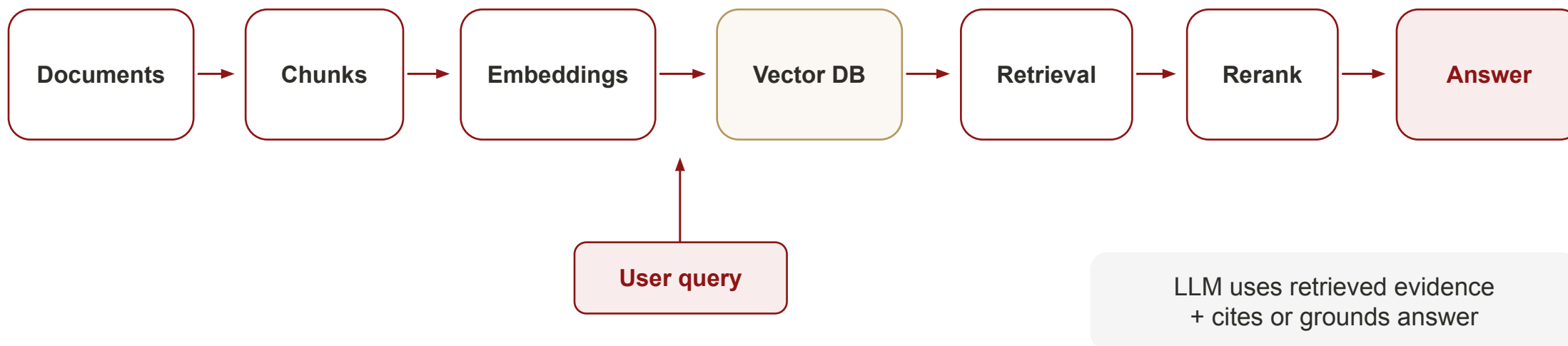
GRPO estimates advantage
from grouped samples

Common in reasoning RL
for math/code-style tasks

Key idea: When answers can be checked, reward signals become cheaper, sharper, and more scalable.

Retrieval-Augmented Generation (RAG)

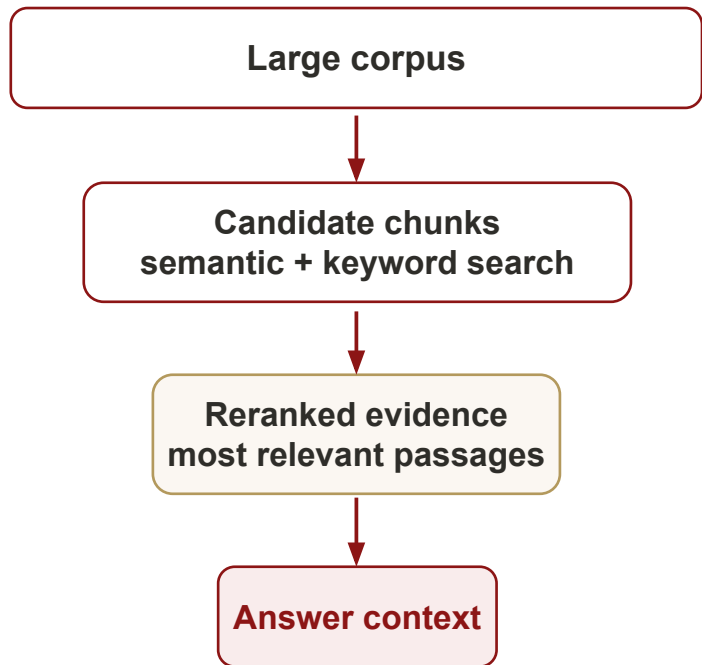
RAG gives an LLM external memory by retrieving relevant documents at answer time.



Key idea: RAG shifts some knowledge from model weights to searchable external data.

RAG quality depends on retrieval design

RAG failures often come from missing, stale, or poorly ranked context—not from generation alone.



DESIGN LEVERS

Chunking
size, overlap, structure

Embeddings
domain fit and language

Hybrid search
semantic + lexical

Reranking
precision before context

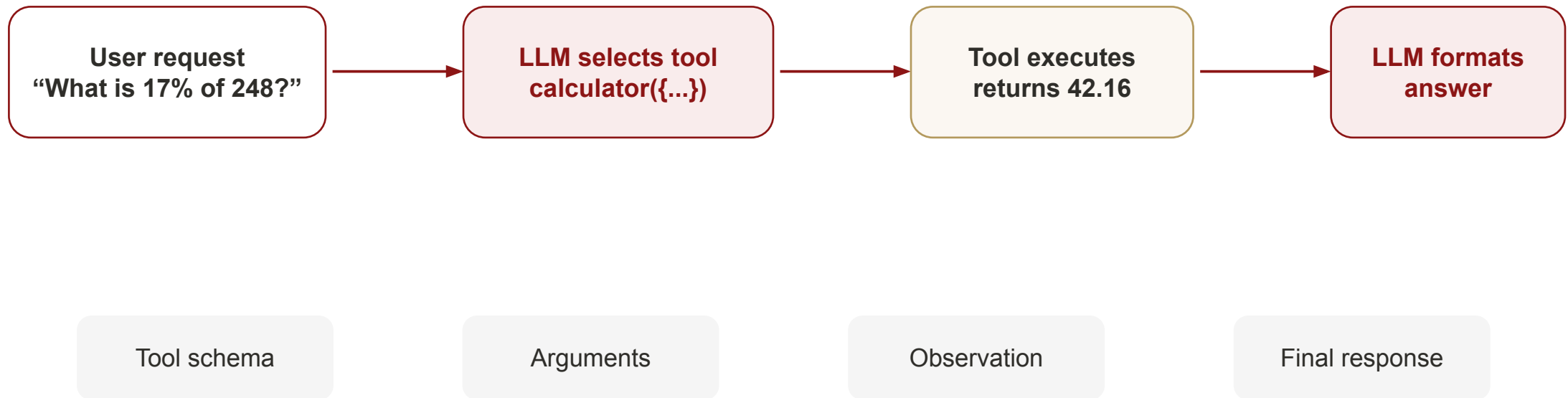
Grounding
citations and refusal

Freshness
index update strategy

Key idea: A good RAG system retrieves the right evidence before asking the LLM to write.

Tool calling: LLMs interact with external functions

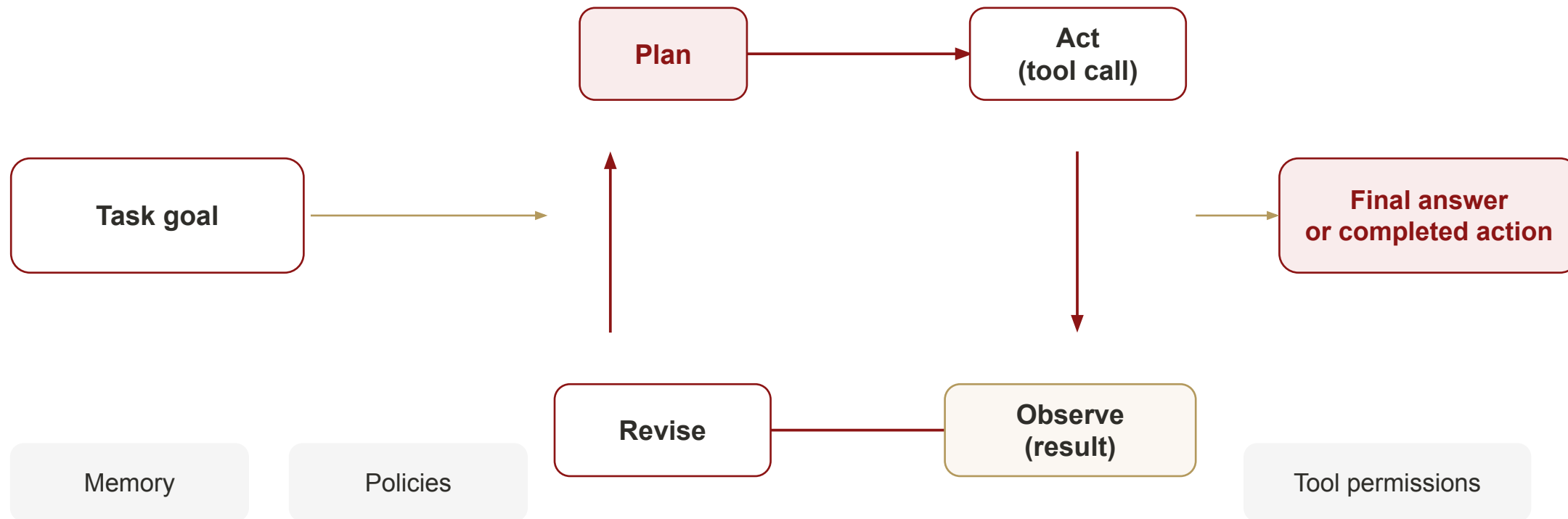
Tools let a model delegate precise or external tasks: search, calculation, databases, calendars, code, APIs.



Key idea: Tool use turns language models from text predictors into interfaces for actions and data.

Agents: plan, act, observe, revise

An agent wraps an LLM in a control loop that can use tools, memory, and feedback over multiple steps.



Key idea: Agent quality is control-loop quality: planning, tool use, state management, and safety boundaries.

LLM evaluation: measure capability and behavior

Evaluation asks whether the system works for the intended users, tasks, and safety constraints.

Knowledge
factual recall
and synthesis

Reasoning
math, logic,
multi-step tasks

Coding
unit tests, repair,
agentic coding

Instruction following
format, constraints,
style

Safety
policy compliance,
refusal quality

Robustness
adversarial prompts,
edge cases

Benchmarks

Human eval

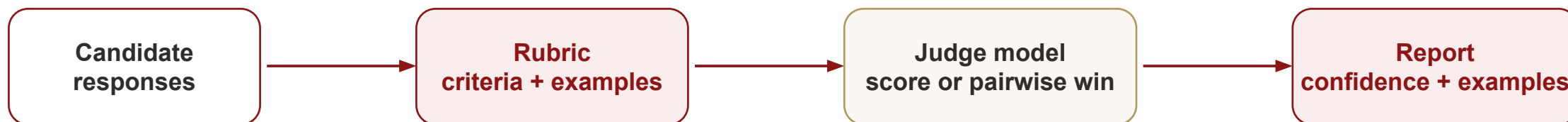
LLM-as-a-judge

Production telemetry

Key idea: Good evaluation blends benchmarks, human judgment, automated checks, and real-world monitoring.

LLM-as-a-judge: scalable but needs calibration

A judge model can score outputs using a rubric, but the evaluation system must guard against bias and leakage.



Calibrate against
human labels

Blind model identity
and randomize order

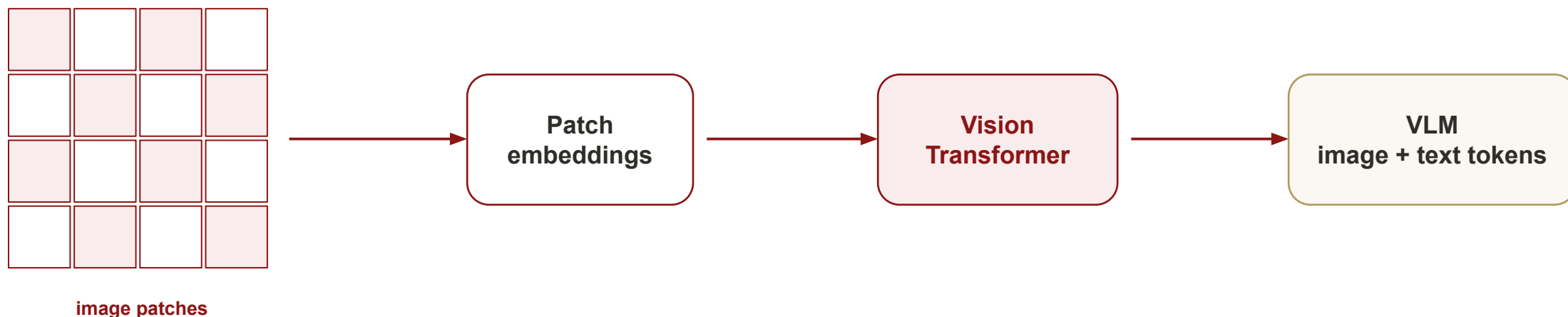
Watch for prompt leakage
and test contamination

Audit failures, not only
average scores

Key idea: Evaluation is itself a system: design rubrics, sampling, judging, and audits deliberately.

Beyond text: Vision Transformers and VLMs

The transformer idea extends to images by treating patches as tokens and aligning visual tokens with language.



ViT: self-attention over image patches

Contrastive or generative alignment connects pixels and words

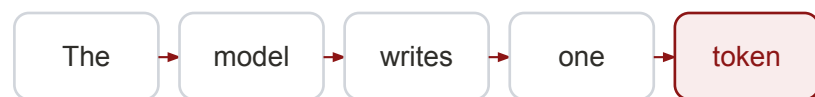
VLMs can caption, answer visual questions, and reason over images

Key idea: Multimodal models reuse the token-and-transformer recipe across different data types.

Diffusion LLMs: generate by iterative denoising

Masked diffusion language models generate text by repeatedly filling or refining masked tokens, not strictly left to right.

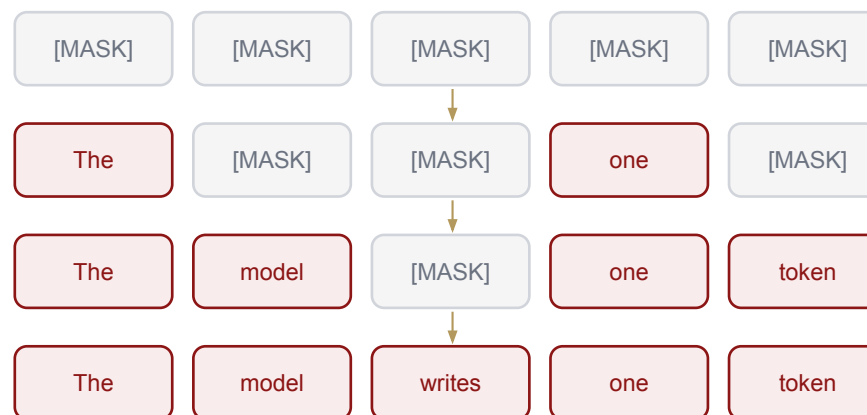
AUTOREGRESSIVE



Left-to-right
next-token loop

Potential advantages
parallel refinement • flexible infilling

MASKED DIFFUSION



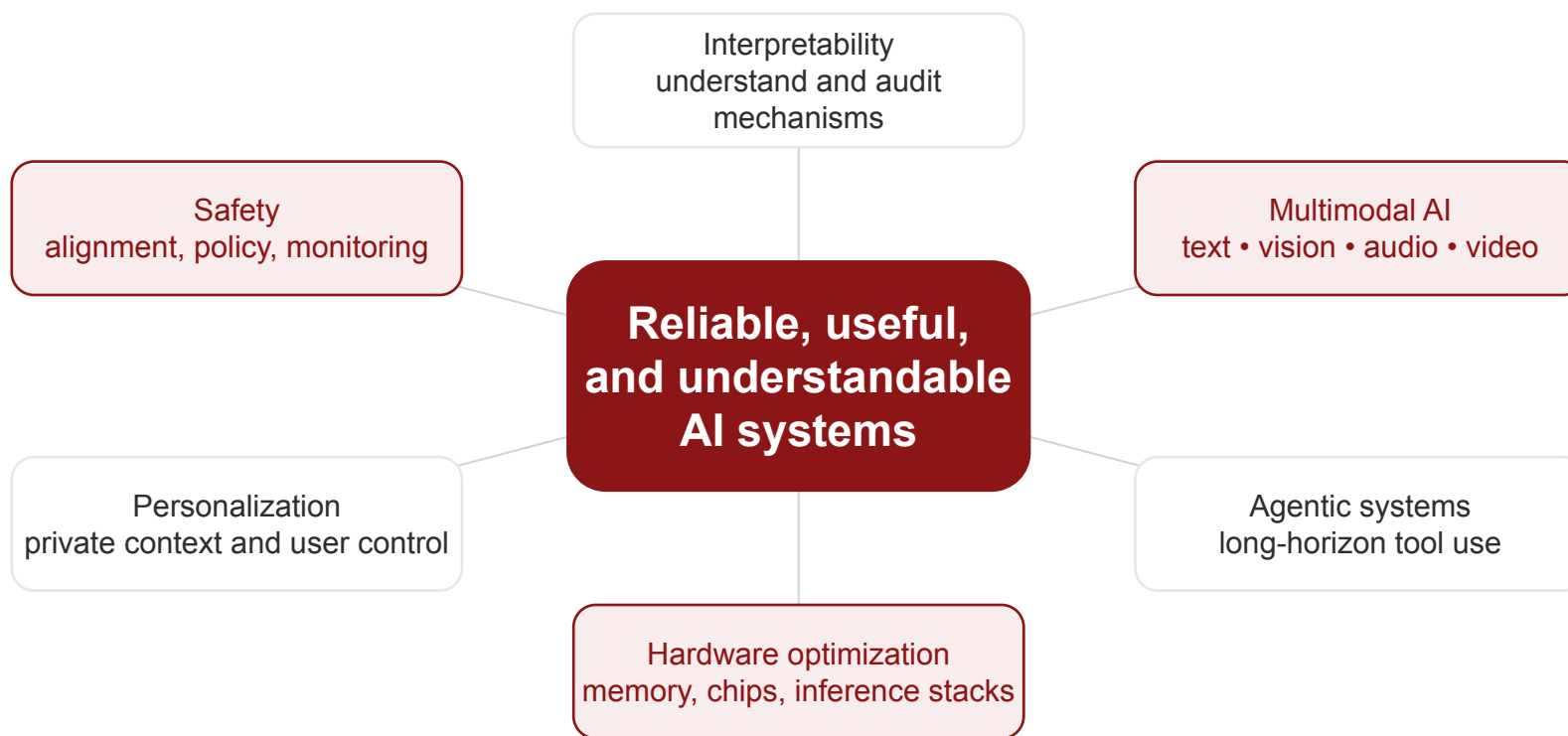
Iterative denoising
can revise many positions

Open challenges
latency • sampling quality • scaling

Key idea: Diffusion LLMs challenge the assumption that strong language generation must be strictly autoregressive.

Future directions: where the field is moving

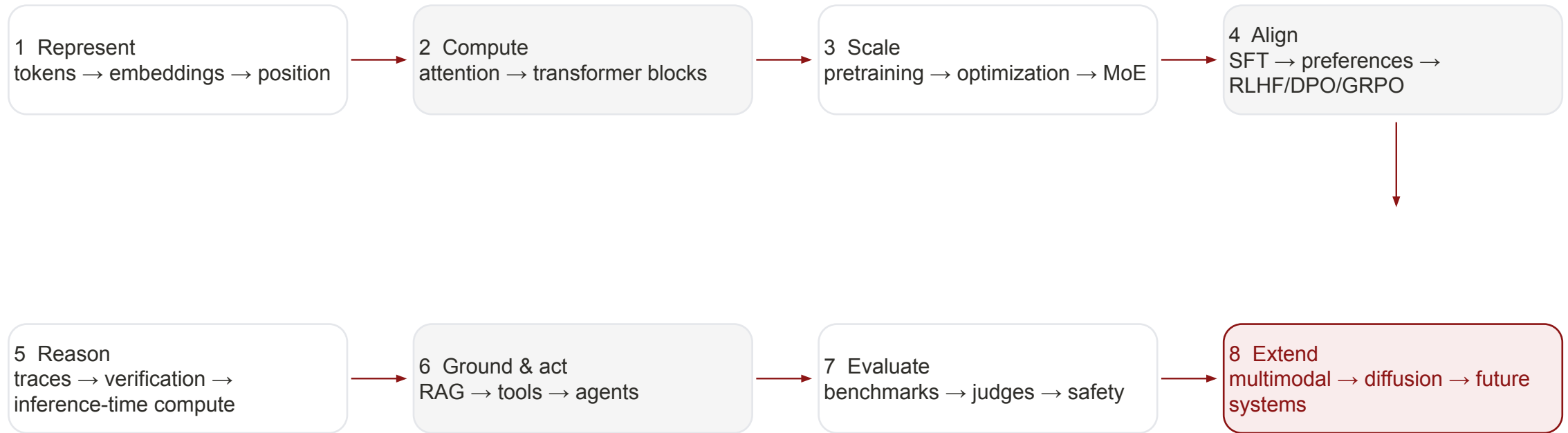
The next generation of LLM systems will be shaped by model capabilities, infrastructure, governance, and human needs.



Key idea: Future progress depends on building complete systems—not just larger models.

Final summary: the full LLM stack in one diagram

Every topic in the course fits into a single pipeline from representation to deployed AI systems.



Course takeaway

Transformers are a reusable computation pattern; LLMs become useful when trained, aligned, grounded, tooled, evaluated, and governed as systems.

Thank you for your attention!
